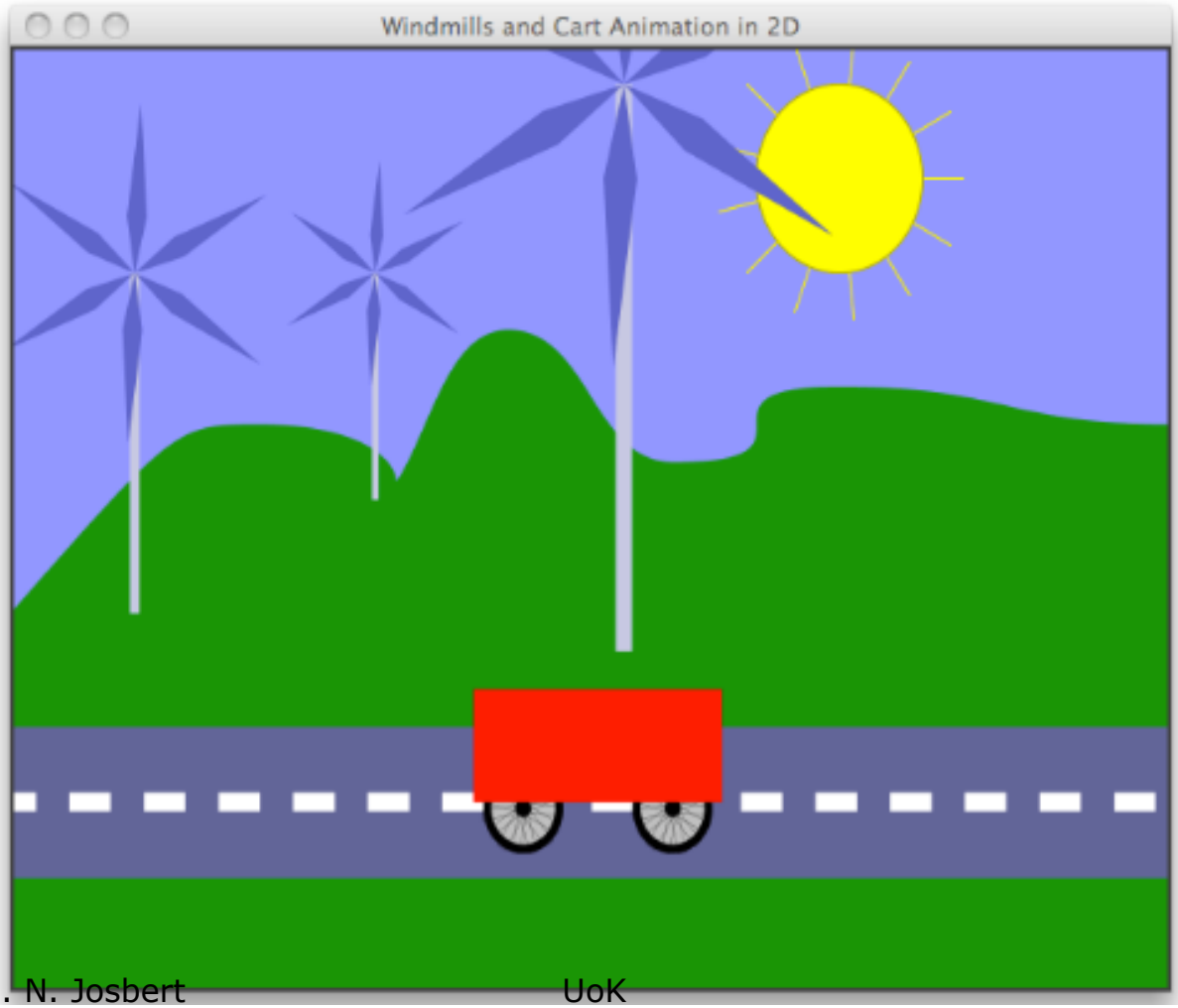
- Topic of the session: Advanced Java Programming
- UNIT 4: 2D SHAPES; COLOUS, DISPLAYING IMAGES
- **University of Kigali (UoK)**

- Trainer name: Dr. NTEZIRIZA NKERABAHIZI Josbert

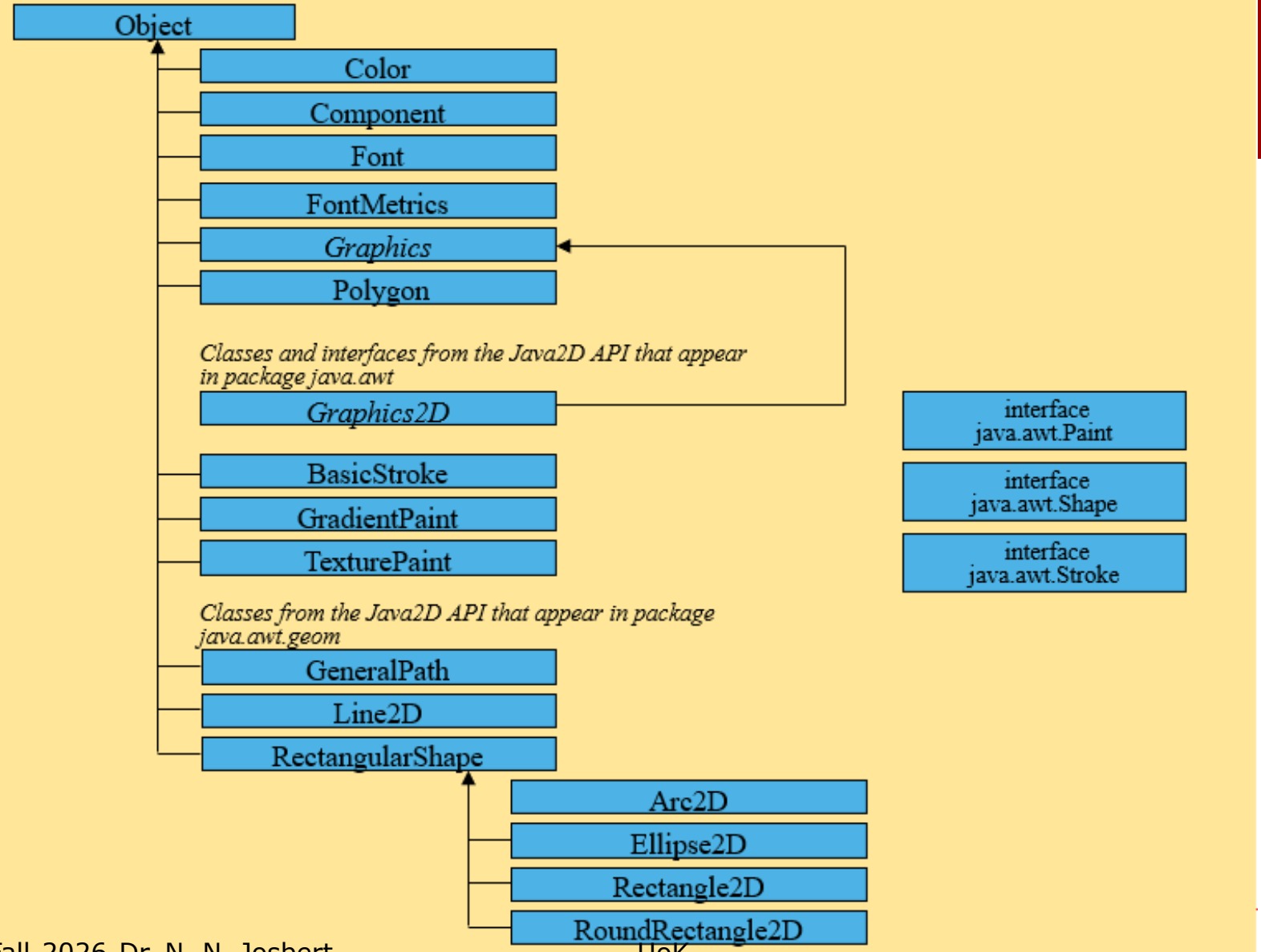
2D SHAPES; COLOUS, DISPLAYING IMAGES



4. 1. Introduction

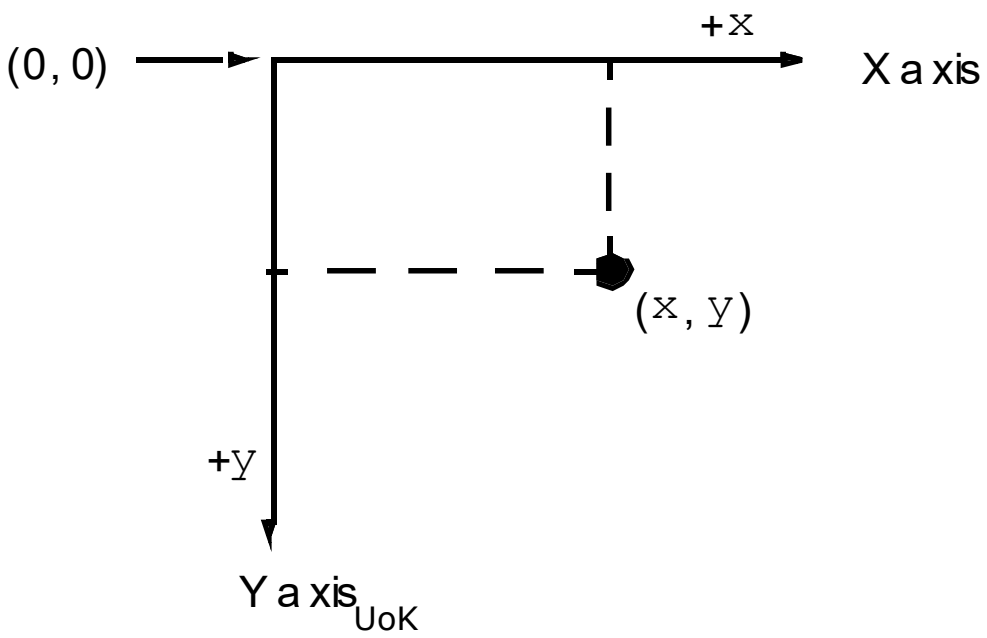
- Java's graphics capabilities
  - ❖Drawing 2D shapes
  - ❖Controlling colors
  - ❖Controlling fonts
- Java 2D API
  - ❖ More sophisticated graphics capabilities
    - ❖Drawing custom 2D shapes
    - ❖Filling shapes with colors and patterns
- Using graphics and GUI classes helps to demonstrate the effectiveness of reusable code.

4. 1. Introduction



4. 1. Introduction

- Java's coordinate system
  - Scheme for identifying all points on screen.
  - Upper-left corner of frame, applet, component has coordinates (0,0).
  - Coordinate point composed of x-coordinate and y-coordinate.



4. 2. Graphics Contexts and Graphics Objects

- **Graphics context:**
  - Enables drawing on screen.
  - Graphics object manages graphics context.
    - ❖ Controls how information is drawn.
  - Class Graphics is abstract:
    - ❖ So you cannot create its object directly.

```
Graphics g = new Graphics(); // ERROR ✗
```

- ❖ Cannot be instantiated: You **cannot create an object** of that class using the new keyword.
- ❖ Contributes to Java's portability.

4. 2. Graphics Contexts and Graphics Objects

– Class Component method paint takes Graphics object

public void paint(Graphics g)

```
public void paint(Graphics g)
```

`paint(...)` → a method used to **draw graphics**

`Graphics g` → an object of **Graphics**

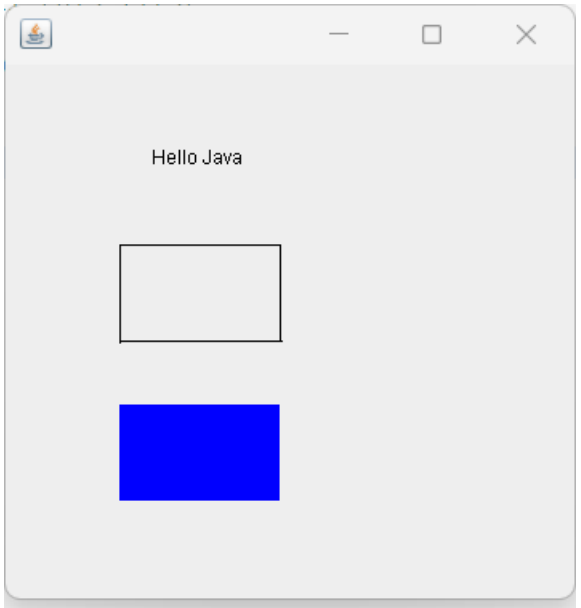
`g` is **provided automatically by Java** (you don't create it)

– Called through method repaint.

➢ You **do NOT call paint() directly**
Instead, you call repaint()

4. 2. Graphics Contexts and Graphics Objects


```java
package Dr_Josbert_2D;

import java.awt.*;
import javax.swing.*;

public class MyFrame extends JFrame {

    public void paint(Graphics g) {
        super.paint(g); // always call this first

        g.drawString("Hello Java", 100, 100);
        g.drawRect(80, 150, 100, 60);
        g.setColor(Color.BLUE);    // set color to blue
        g.fillRect(80, 250, 100, 60); // draw filled rectangle
    }

    public static void main(String[] args) {
        MyFrame f = new MyFrame();
        f.setSize(300, 300);
        f.setVisible(true);
    }
}
```



4. 3. Color Control

- **Class Color:**
  - Defines methods and constants for manipulating colors
  - Constants (Color.YELLOW)
  - Colors are also created from Red, Green, and Blue components
    - RGB values (0-255)
  - JColorChooser class available in javax.swing
    - GUI component that allows user to select color.
    - JColorChooser(ref to parent component, title bar string, initial selected color).

4. 3. Color Control

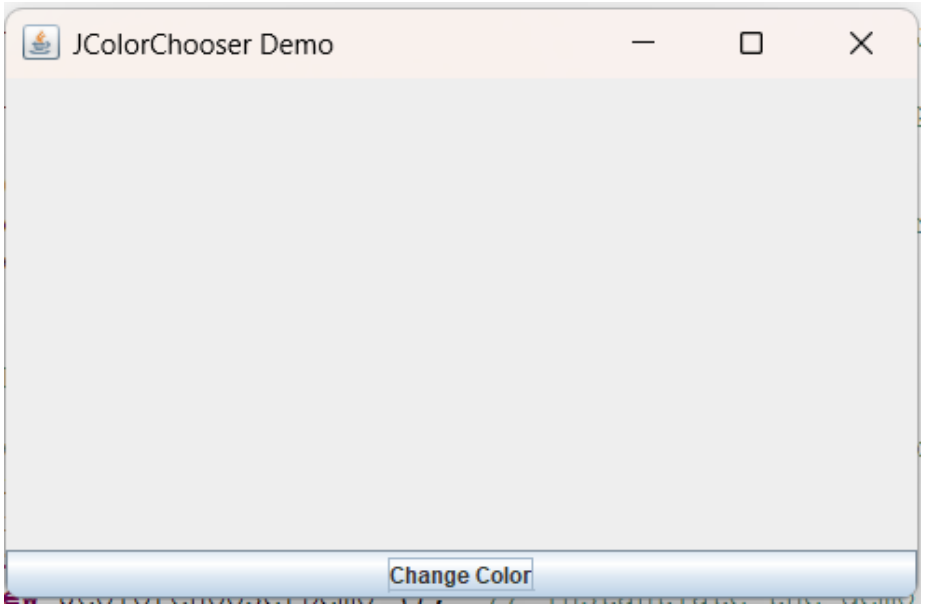
- Examples

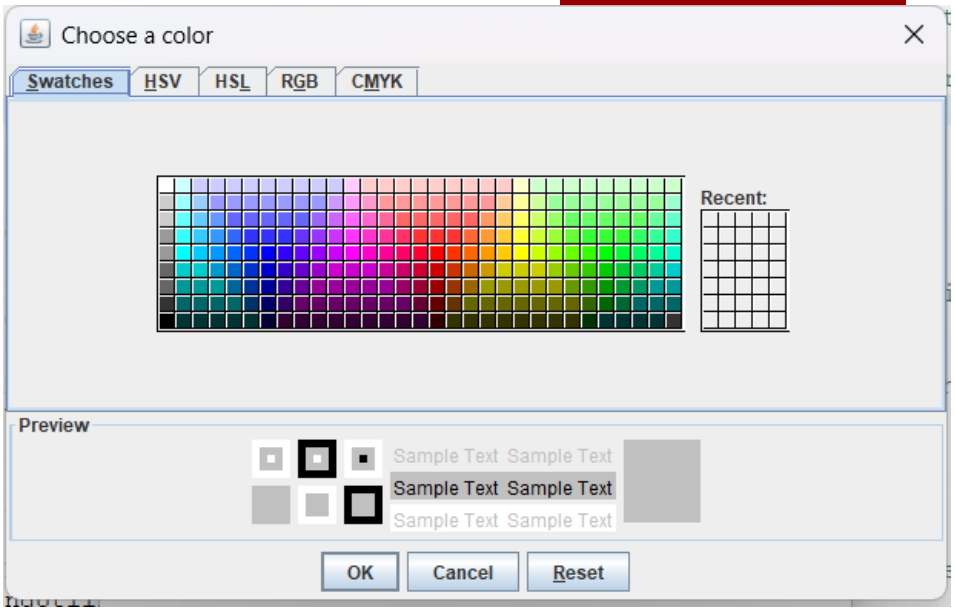
```
1  package Dr_Josbert_2D;
2  import java.awt.*;          // Importing AWT classes for Graphics, Color, and Layout management
3  import java.awt.event.*;    // Importing AWT event classes and listener interfaces for
4                                  //handling user interactions
5  import javax.swing.*;       // Importing Swing classes for building the GUI components
6  @SuppressWarnings("serial") // Suppressing serial warning for serialization
7  public class JColorChooserDemo_ extends JFrame { // Class extending JFrame to create a windowed application
8
9      JPanel panel; // Panel to hold components and set background color
10     Color bgColor = Color.LIGHT_GRAY;  // Initial color for the panel's background
11
12     // Constructor to setup the UI components and event handlers
13     public JColorChooserDemo_() {
14         panel = new JPanel(new BorderLayout()); // Creating a JPanel with BorderLayout
15
16         JButton btnColor = new JButton("Change Color"); // Button to trigger color change
17         panel.add(btnColor, BorderLayout.SOUTH); // Adding button to the southern part of the panel
18
19         // Adding an action listener to the button for handling click events
20         btnColor.addActionListener(new ActionListener() {
21             @Override
22             public void actionPerformed(ActionEvent evt) {
23                 // Open a color chooser dialog to let the user select a color
24                 Color color = JColorChooser.showDialog(JColorChooserDemo_.this,
25                         "Choose a color", bgColor);
26
27                 if (color != null) { // Check if the user selected a new color
28                     bgColor = color; // Update the background color with the new selection
29                 }
30                 panel.setBackground(bgColor); // Set the background color of the panel
31                 //to the selected color
32             }
33         });
```

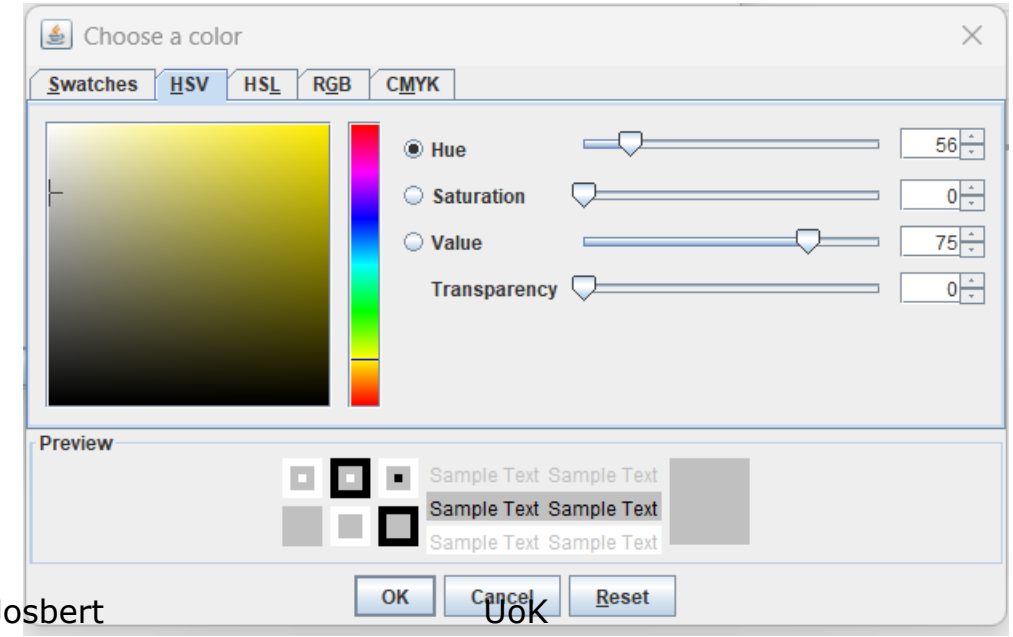
4. 3. Color Control


```
34
35          setContentPane(panel); // Setting the content pane of the JFrame to our panel
36
37          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Exit application on close
38          setTitle("JColorChooser Demo"); // Set title of the window
39          setSize(300, 200); // Setting the size of the window
40          setLocationRelativeTo(null);  // Center the application window on the screen
41          setVisible(true);             // Make the window visible
42      }
43
44      // The entry point of the program
45      public static void main(String[] args) {
46          // Run the GUI codes inside the Event-Dispatching thread for thread safety
47          SwingUtilities.invokeLater(new Runnable() {
48              @Override
49              public void run() {
50                 new JColorChooserDemo_();  // Instantiate the demo class, triggering the UI setup
51              }
52          });
53      }
54
55  }
56
```

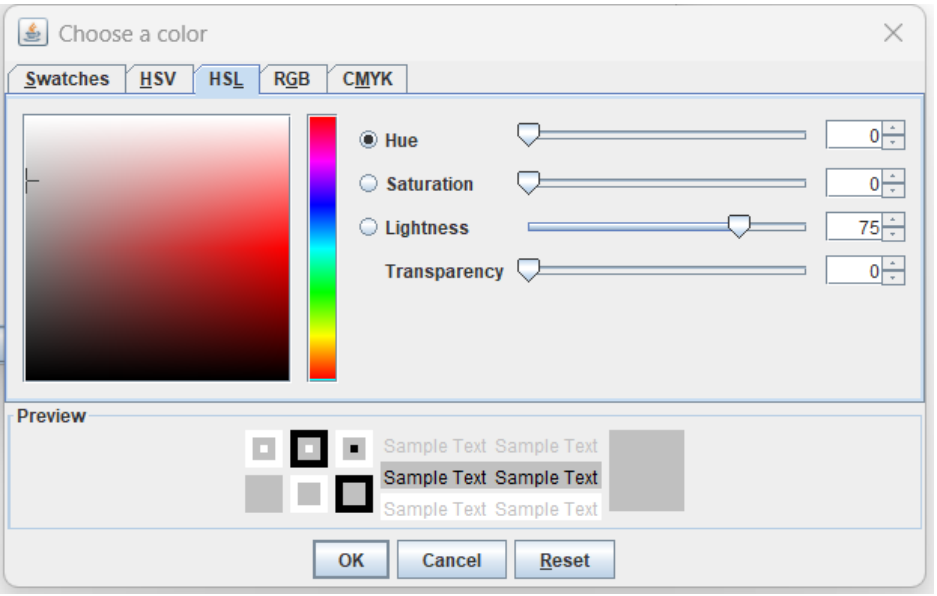
4. 3. Color Control

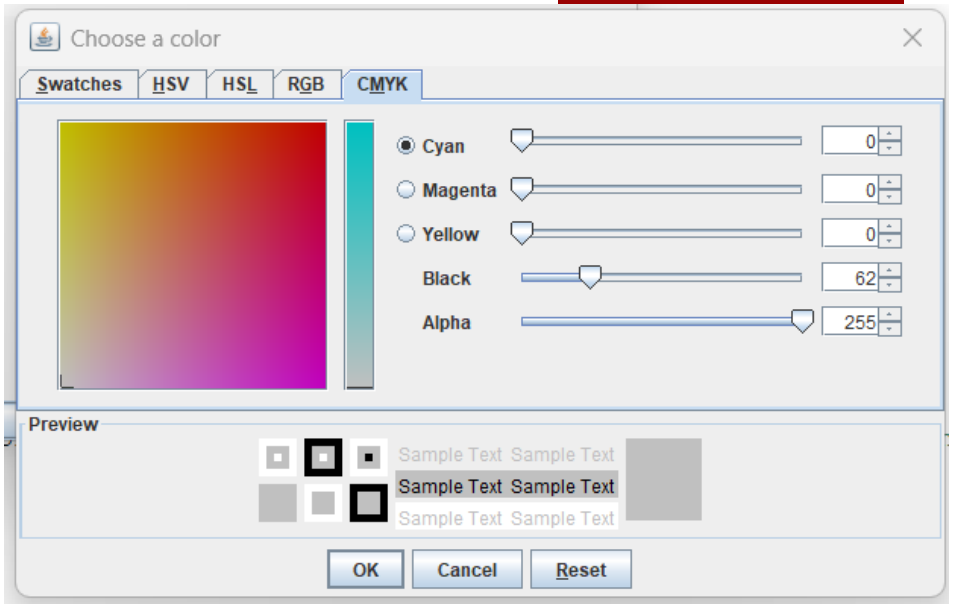


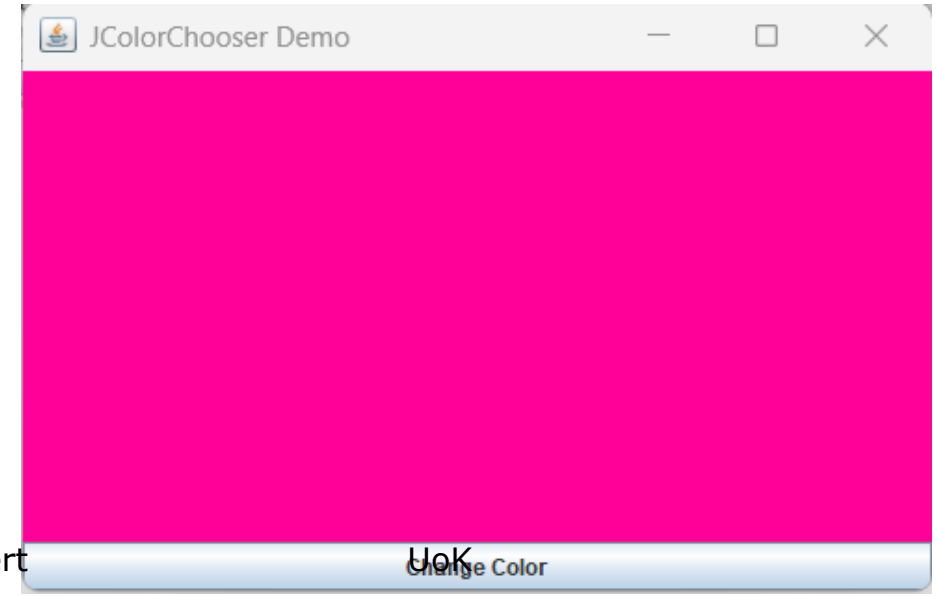




4. 3. Color Control







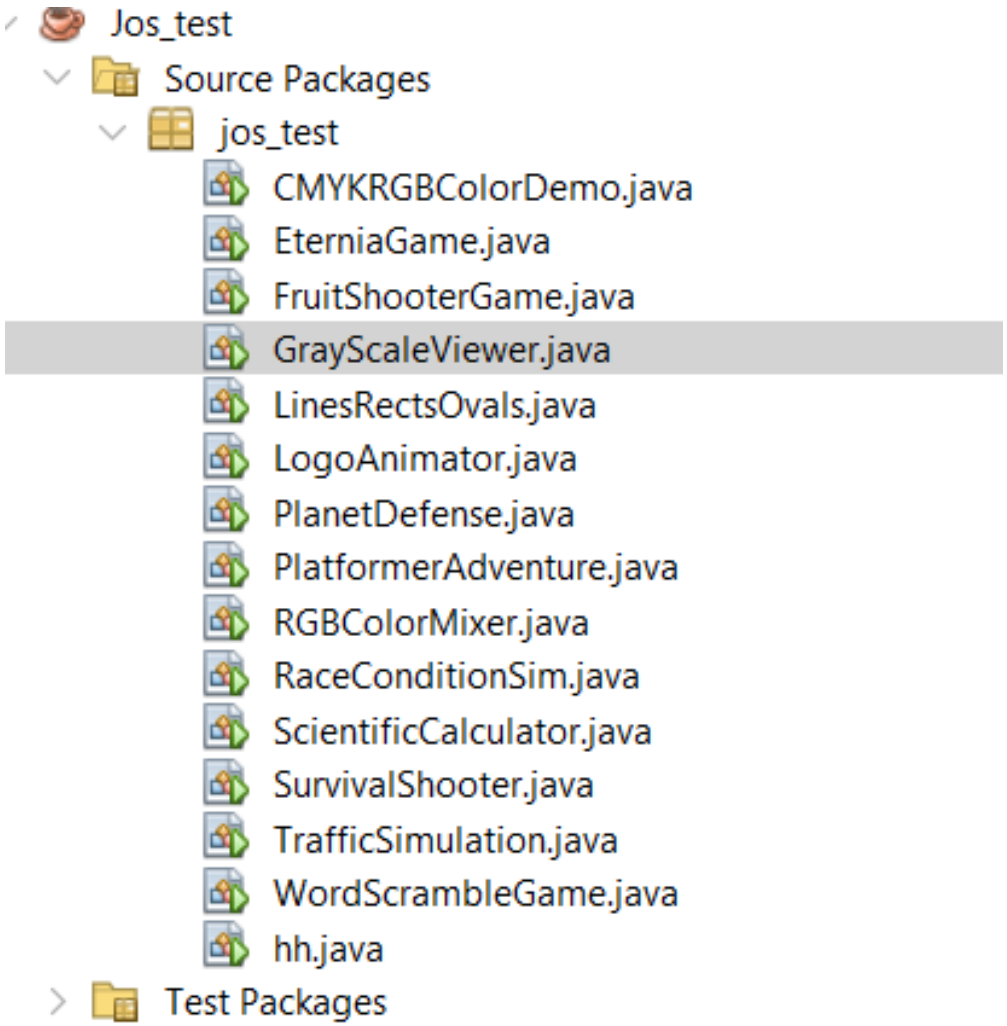
4. 3. Color Control

Color constant and value

Color constant	Color	RGB value
public final static Color ORANGE	orange	255, 200, 0
public final static Color PINK	pink	255, 175, 175
public final static Color CYAN	cyan	0, 255, 255
public final static Color MAGENTA	magenta	255, 0, 255
public final static Color YELLOW	yellow	255, 255, 0
public final static Color BLACK	black	0, 0, 0
public final static Color WHITE	white	255, 255, 255
public final static Color GRAY	gray	128, 128, 128
public final static Color LIGHT_GRAY	light gray	192, 192, 192
public final static Color DARK_GRAY	dark gray	64, 64, 64
public final static Color RED	red	255, 0, 0
public final static Color GREEN	green	0, 255, 0
public final static Color BLUE	blue	0, 0, 255

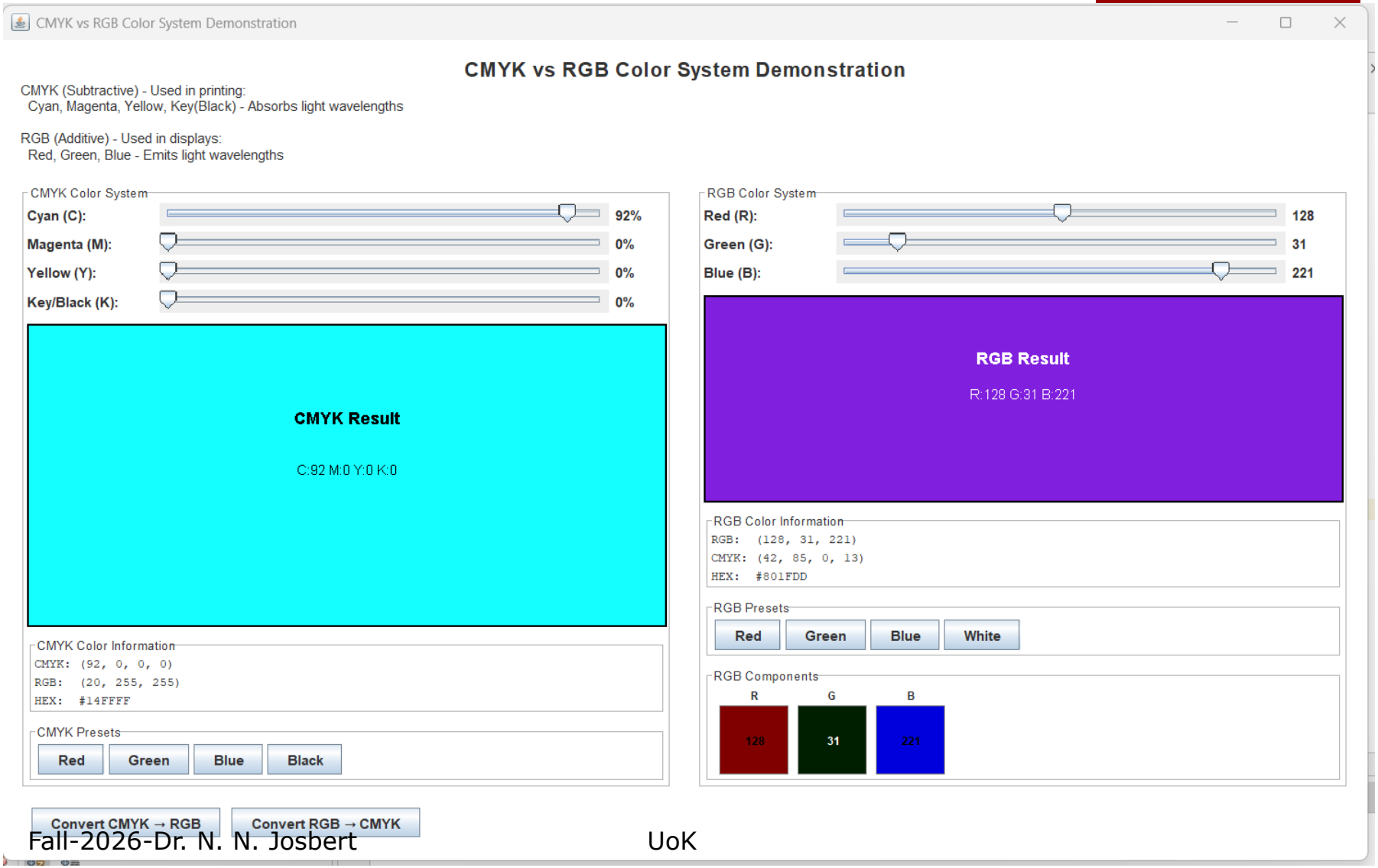
4. 3. Color Control

Color constant and value

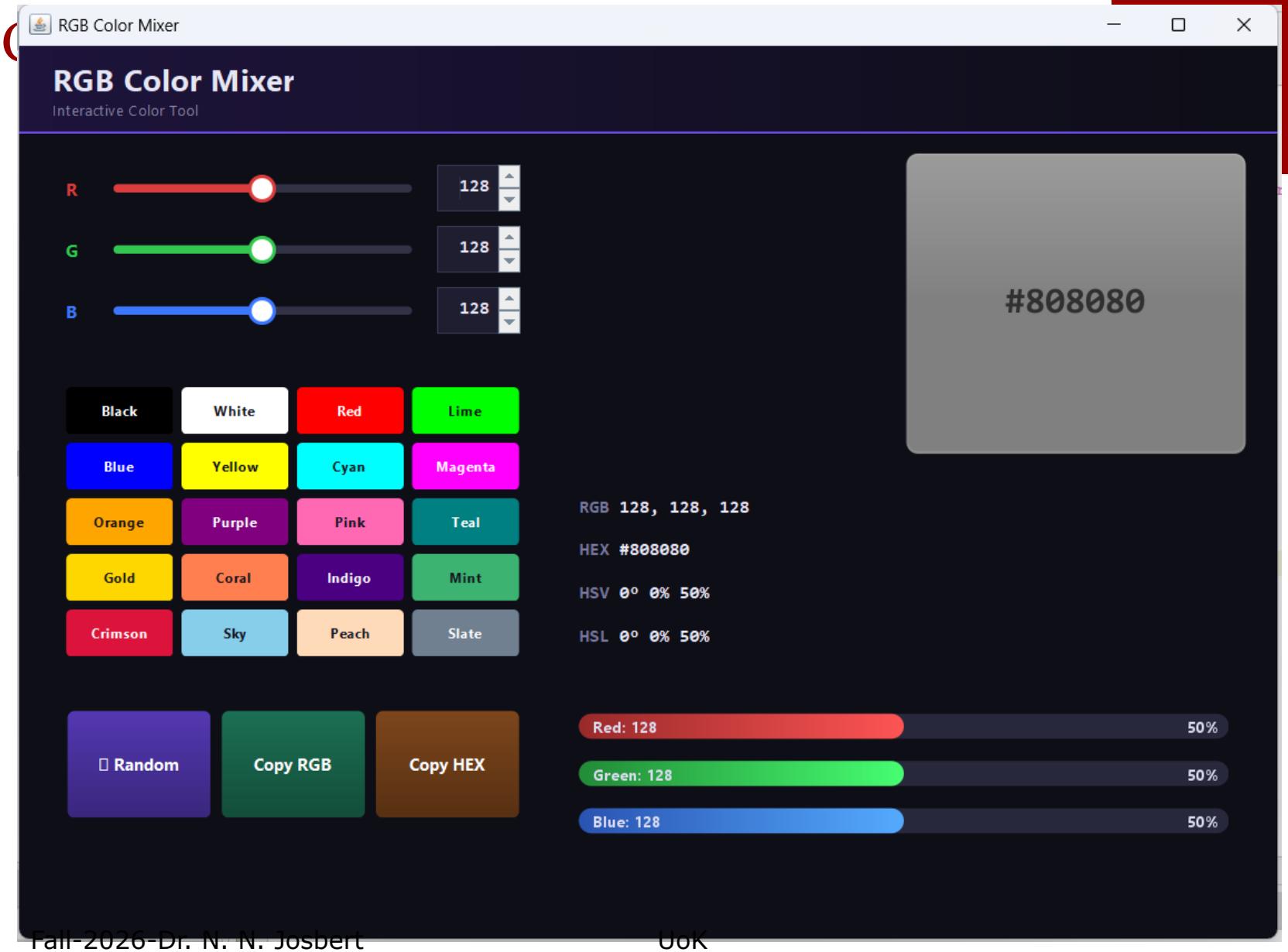


4. 3. Color Control

Color constant and value

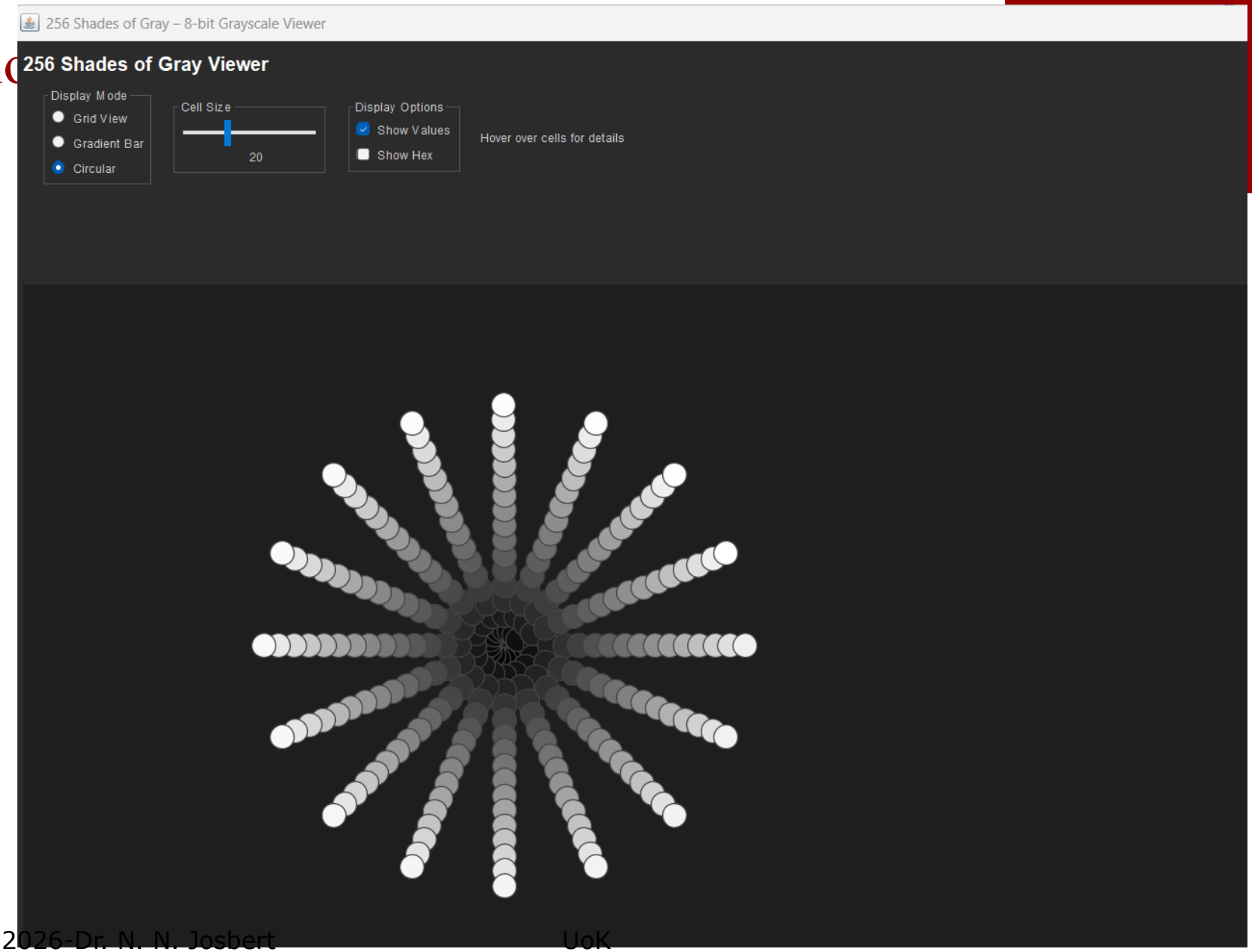


4. 3. Color Control



4. 3. Color Control

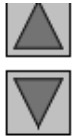
Colo



4. 3. Color Control

Color methods & color-related Graphic methods

Method	Description
Color constructors and methods	
`public Color( int r, int g, int b )`	
	Creates a color based on red, green and blue components expressed as integers from 0 to 255.
`public Color( float r, float g, float b )`	
	Creates a color based on red, green and blue components expressed as floating-point values from 0.0 to 1.0.
`public int getRed()`	
	Returns a value between 0 and 255 representing the red content.
`public int getGreen()`	
	Returns a value between 0 and 255 representing the green content.
`public int getBlue()`	
	Returns a value between 0 and 255 representing the blue content.
Graphics methods for manipulating Colors	
`public Color getColor()`	
	Returns a `Color` object representing the current color for the graphics context.
`public void setColor( Color c )`	
	Sets the current color for drawing with the graphics context.



Outline

ShowColors.java

Line 18

Line 24

Line 25

Line 26

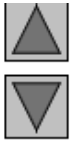
```
1  // Fig. 12.5: ShowColors.java
2  // Demonstrating Colors.
3  import java.awt.*;
4  import javax.swing.*;
5
6  public class ShowColors extends JFrame {
7
8     // constructor sets window's title bar string and dimensions
9     public ShowColors()
10    {
11       super( "Using colors" );
12
13       setSize( 400, 130 );
14       setVisible( true );
15    }
16
17    // draw rectangles and Strings in different colors
18    public void paint( Graphics g )
19    {
20       // call superclass's paint method
21       super.paint( g );
22
23       // set new drawing color using integers
24       g.setColor( new Color( 255, 0, 0 ) );
25       g.fillRect( 25, 25, 100, 20 );
26       g.drawString( "Current RGB: " + g.getColor(), 130, 40 );
27
```

Paint window when application begins execution

Method `setColor` sets color's RGB value

Method `fillRect` creates filled rectangle at specified coordinates using current RGB value

Method `drawString` draws colored text at specified coordinates



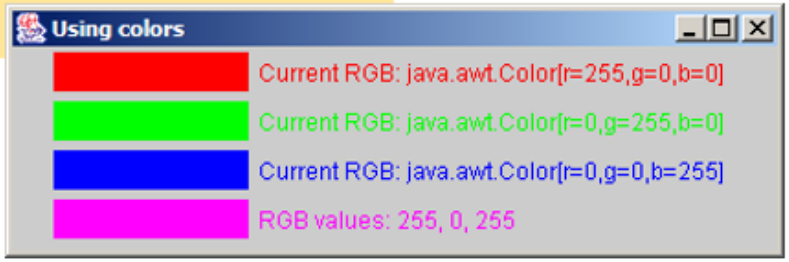
Outline

ShowColors.java

Lines 34-39

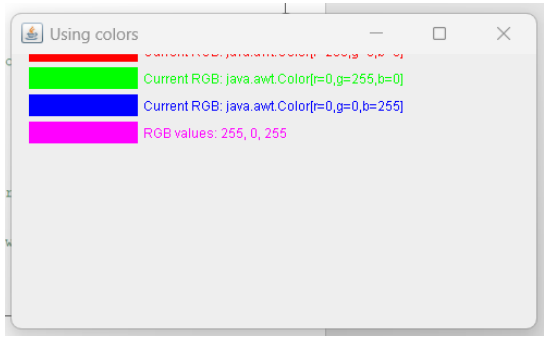
```
28        // set new drawing color using floats
29        g.setColor( new Color( 0.0f, 1.0f, 0.0f ) );
30        g.fillRect( 25, 50, 100, 20 );
31        g.drawString( "Current RGB: " + g.getColor(), 130, 65 );
32
33        // set new drawing color using static Color objects
34        g.setColor( Color.BLUE );
35        g.fillRect( 25, 75, 100, 20 );
36        g.drawString( "Current RGB: " + g.getColor(), 130, 90 );
37
38        // display individual RGB values
39        Color color = Color.MAGENTA;
40        g.setColor( color );
41        g.fillRect( 25, 100, 100, 20 );
42        g.drawString( "RGB values: " + color.getRed() + ", " +
43           color.getGreen() + ", " + color.getBlue(), 130, 115 );
44
45     } // end method paint
46
47     // execute application
48     public static void main( String args[] )
49     {
50        ShowColors application = new ShowColors();
51        application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
52     }
53
54  } // end class ShowColors
```

Use constant in class `Color` to specify current color



```java
package Dr_Josbert_2D;

//Import AWT package for Graphics and Color classes
import java.awt.*;
//Import Swing package for JFrame
import javax.swing.*;

//ShowColors class extends JFrame to create a window that demonstrates color usage
public class ShowColors extends JFrame {

 // Constructor sets the window's title bar string and dimensions
 public ShowColors() {
     // Call JFrame constructor with the window title
     super("Using colors");

     // Set the width (400px) and height (130px) of the window
     setSize(400, 130);

     // Make the window visible on screen
     setVisible(true);
 }

 // Override paint method to draw rectangles and Strings in different colors
 public void paint(Graphics g) {
     // Call superclass's paint method to ensure proper rendering of the frame
     super.paint(g);

     // --- Example 1: Set color using integer RGB values (red = 255, green = 0, blue = 0) ---
     g.setColor(new Color(255, 0, 0));
     // Draw a filled red rectangle at position (25, 25) with width 100 and height 20
     g.fillRect(25, 25, 100, 20);
     // Display the current RGB color values as a string next to the rectangle
     g.drawString("Current RGB: " + g.getColor(), 130, 40);

```

```
35      // ---GREEN COLOR Example 2: Set color using float RGB values (0.0-1.0 range), here pure green ---
36      g.setColor(new Color(0.0f, 1.0f, 0.0f));
37      // Draw a filled green rectangle at position (25, 50)
38      g.fillRect(25, 50, 100, 20);
39      // Display the current RGB color values as a string next to the rectangle
40      g.drawString("Current RGB: " + g.getColor(), 130, 65);
41
42      // --- Example 3: Set color using a predefined static Color constant (Color.BLUE) ---
43      g.setColor(Color.BLUE);
44      // Draw a filled blue rectangle at position (25, 75)
45      g.fillRect(25, 75, 100, 20);
46      // Display the current RGB color values as a string next to the rectangle
47      g.drawString("Current RGB: " + g.getColor(), 130, 90);
48
49      // --- Example 4: Use a Color object to access individual RGB components ---
50      // Create a Color object using the predefined MAGENTA constant
51      Color color = Color.MAGENTA;
52      g.setColor(color);
53      // Draw a filled magenta rectangle at position (25, 100)
54      g.fillRect(25, 100, 100, 20);
55      // Display the individual red, green, and blue components of the color
56      g.drawString("RGB values: " + color.getRed() + ", " +
57              color.getGreen() + ", " + color.getBlue(), 130, 115);
58
59  } // end method paint
60
61  // Main method - entry point to execute the application
62  public static void main(String[] args) {
63      // Create an instance of ShowColors, which triggers the constructor
64      // and displays the window
65      ShowColors app = new ShowColors();
66      // Set the default close operation so the app exits when the window is closed
67      app.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
68  }
69
70  } // end class ShowColors
```



```
1  // Fig. 12.6: ShowColors2.java
2  // Choosing colors with JColorChooser.
3  import java.awt.*;
4  import java.awt.event.*;
5  import javax.swing.*;
6
7  public class ShowColors2 extends JFrame {
8     private JButton changeColorButton;
9     private Color color = Color.LIGHT_GRAY;
10    private Container container;
11
12    // set up GUI
13    public ShowColors2()
14    {
15       super( "Using JColorChooser" );
16
17       container = getContentPane();
18       container.setLayout( new FlowLayout() );
19
20       // set up changeColorButton and register its event handler
21       changeColorButton = new JButton( "Change Color" );
22       changeColorButton.addActionListener(
23
```

```
24          new ActionListener() {  // anonymous inner class
25
26             // display JColorChooser when user clicks button
27             public void actionPerformed( ActionEvent event )
28             {
29                color = JColorChooser.showDialog(
30                   ShowColors2.this, "Choose a color", color );
31
32                // set default color, if no color is returned
33                if ( color == null )
34                   color = Color.LIGHT_GRAY;
35
36                // change content pane's background color
37                container.setBackground( color );
38             }
39
40          } // end anonymous inner class
41
42       ); // end call to addActionListener
43
44       container.add( changeColorButton );
45
46       setSize( 400, 130 );
47       setVisible( true );
48
49    } // end ShowColor2 constructor
50
```

Outline

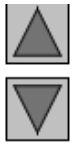
2.jav

`JColorChooser` allows user to choose from among several colors

Line 29

`static` method `showDialog` displays the color chooser dialog

```
51    // execute application
52    public static void main( String args[] )
53    {
54       ShowColors2 application = new ShowColors2();
55       application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
56    }
57
58 } // end class ShowColors2
```



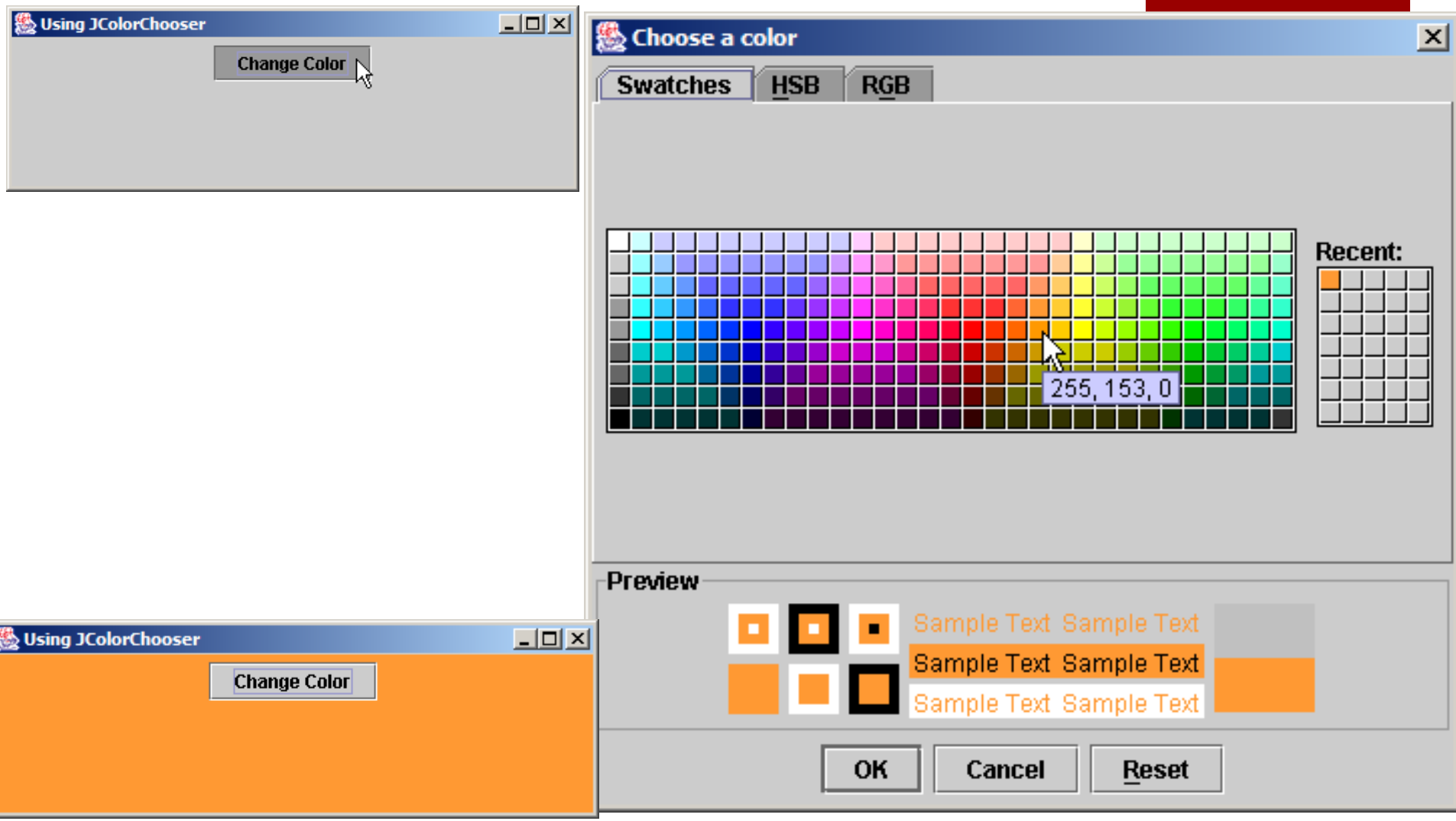
Outline

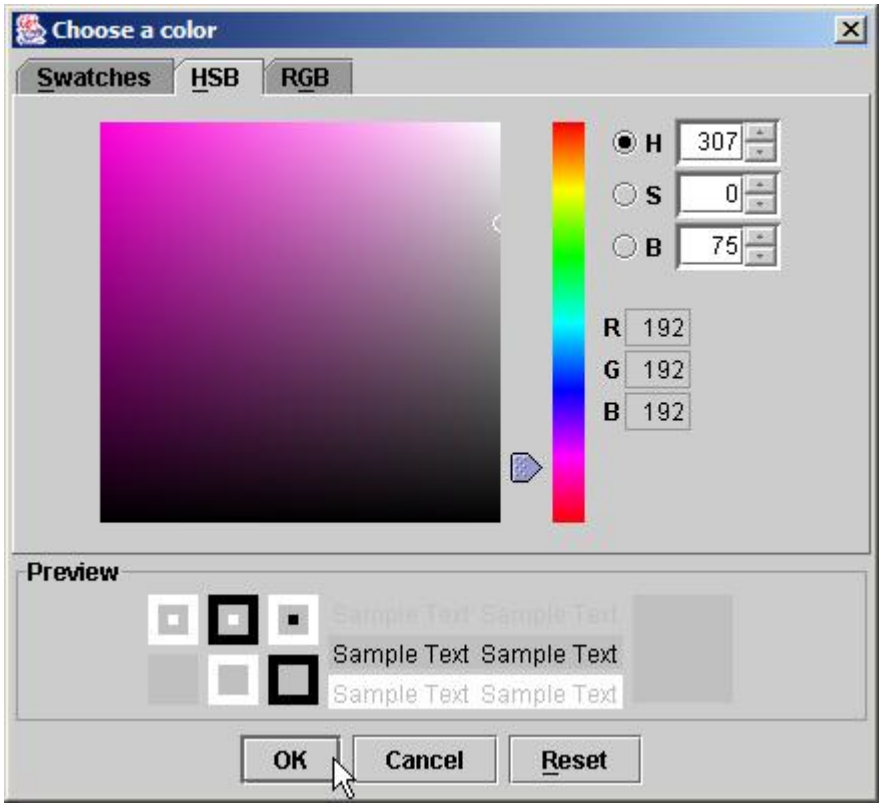
showColors2.java

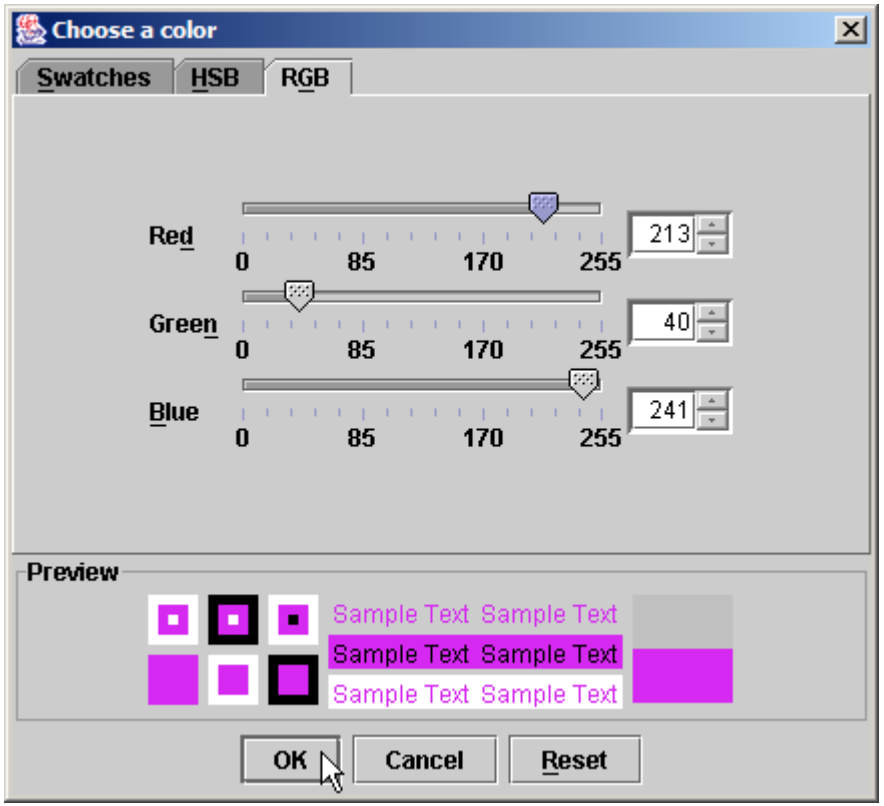
```java
package Dr_Josbert_2D;

//Fig. 12.6: ShowColors2.java
//Choosing colors with JColorChooser.

//Import AWT package for Color, Container, and FlowLayout classes
import java.awt.*;
//Import AWT event package for ActionListener and ActionEvent
import java.awt.event.*;
//Import Swing package for JFrame, JButton, and JColorChooser
import javax.swing.*;

//ShowColors2 class extends JFrame to create a window with a color chooser dialog
public class ShowColors2 extends JFrame {

// Button that the user clicks to open the color chooser dialog
private JButton changeColorButton;

// Stores the currently selected color, initialized to light gray
private Color color = Color.LIGHT_GRAY;

// Container holds all the GUI components inside the JFrame
private Container container;

// Constructor: sets up the GUI window and its components
public ShowColors2() {
   // Set the title of the window's title bar
   super("Using JColorChooser");

   // Get the content pane of the JFrame to add components to
   container = getContentPane();

   // Set FlowLayout so components are arranged left-to-right
   container.setLayout(new FlowLayout());

   // Create the "Change Color" button that opens the color chooser
   changeColorButton = new JButton("Change Color");
```

```
38
39    // Register an ActionListener to handle button click events
40    changeColorButton.addActionListener(
41
42       // Anonymous inner class that implements ActionListener
43       new ActionListener() {
44
45          // This method is called automatically when the button is clicked
46          public void actionPerformed(ActionEvent event) {
47
48             // Open the JColorChooser dialog and store the chosen color
49             // Arguments: parent window, dialog title, initial color
50             color = JColorChooser.showDialog(
51                ShowColors2.this, "Choose a color", color);
52
53             // If the user closes the dialog without choosing a color,
54             // JColorChooser returns null — default back to light gray
55             if (color == null)
56                color = Color.LIGHT_GRAY;
57
58             // Apply the selected color as the background of the content pane
59             container.setBackground(color);
60
61          } // end method actionPerformed
62
63       } // end anonymous inner class
64
65    ); // end call to addActionListener
66
67    // Add the "Change Color" button to the content pane
68    container.add(changeColorButton);
69
70    // Set the width (400px) and height (130px) of the window
71    setSize(400, 130);
72
73    // Make the window visible on screen
74    setVisible(true);
```

```
75
76 } // end ShowColors2 constructor
77
78 // Main method – entry point to launch the application
79 public static void main(String args[]) {
80
81    // Create an instance of ShowColors2 which builds and displays the GUI
82    ShowColors2 application = new ShowColors2();
83
84    // Exit the application when the user closes the window
85    application.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
86
87 } // end main
88
89 } // end class ShowColors2
```


Using JColorChooser
Change Color
Choose a color
Swatches
HSB
RGB
255, 153, 0
Recent:
Preview
Sample Text Sample Text
Sample Text Sample Text
Sample Text Sample Text
OK
Cancel
Reset
Using JColorChooser
Change Color


Choose a color
Swatches
HSB
RGB
H 307
S 0
B 75
R 192
G 192
B 192
Preview
Sample Text Sample Text
Sample Text Sample Text
Sample Text Sample Text
OK
Cancel
Reset


Choose a color
Swatches
HSB
RGB
Red
0 85 170 255
213
Green
0 85 170 255
40
Blue
0 85 170 255
241
Preview
Sample Text Sample Text
Sample Text Sample Text
Sample Text Sample Text
OK
Cancel
Reset

4.4. Font Control

- Class Font
  - Contains methods and constants for font control.
  - Font constructor takes three arguments.
    - **Font name**
      - ❖Monospaced, SansSerif, Serif, etc.
    - **Font style**
      - ❖Font.PLAIN, Font.ITALIC and Font.BOLD
    - **Font size**
      - Measured in points (1/72 of inch)

4.4. Font Control

Font-related methods and constants

Method or constant	Description
`Font` constants, constructors and methods for drawing polygons	
`public final static int PLAIN`	
	A constant representing a plain font style.
`public final static int BOLD`	
	A constant representing a bold font style.
`public final static int ITALIC`	
	A constant representing an italic font style.
`public Font( String name, int style, int size )`	
	Creates a `Font` object with the specified font, style and size.
`public int getStyle()`	
	Returns an integer value indicating the current font style.
`public int getSize()`	
	Returns an integer value indicating the current font size.
`public String getName()`	
	Returns the current font name as a string.
`public String getFamily()`	
	Returns the font's family name as a string.
`public boolean isPlain()`	
	Tests a font for a plain font style. Returns `true` if the font is plain.
`public boolean isBold()`	
	Tests a font for a bold font style. Returns `true` if the font is bold.
`public boolean isItalic()`	
	Tests a font for an italic font style. Returns `true` if the font is italic.

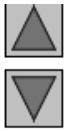
4.4. Font Control

Font-related methods and constants

Method or constant	Description
`Graphics` methods for manipulating `Font`s	
`public Font getFont()`	
	Returns a `Font` object reference representing the current font.
`public void setFont( Font f )`	
	Sets the current font to the font, style and size specified by the `Font` object reference `f`.

4.4. Font Control

Font-related methods and constants



Outline

Fonts.java

Line 24

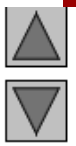
Line 25

```
1  // Fig. 12.9: Fonts.java
2  // Using fonts.
3  import java.awt.*;
4  import javax.swing.*;
5
6  public class Fonts extends JFrame {
7
8     // set window's title bar and dimensions
9     public Fonts()
10    {
11       super( "Using fonts" );
12
13       setSize( 420, 125 );
14       setVisible( true );
15    }
16
17    // display Strings in different fonts and colors
18    public void paint( Graphics g )
19    {
20       // call superclass's paint method
21       super.paint( g );
22
23       // set font to Serif (Times), bold, 12pt and draw a string
24       g.setFont( new Font( "Serif", Font.BOLD, 12 ) );
25       g.drawString( "Serif 12 point bold.", 20, 50 );
```

Method `setFont` sets current font

Draw text using current font

4.4. Font Control



Outline

Fonts.java

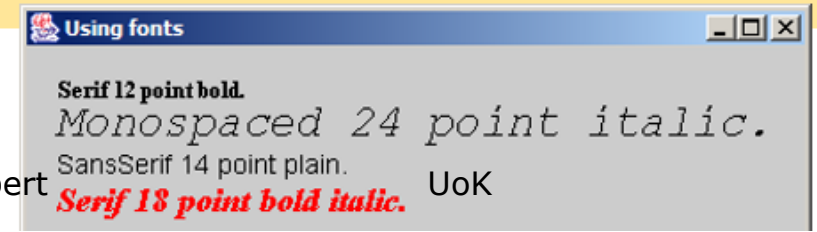
Line 32

Line 37

```
26
27       // set font to Monospaced (Courier), italic, 24pt and draw a string
28       g.setFont( new Font( "Monospaced", Font.ITALIC, 24 ) );
29       g.drawString( "Monospaced 24 point italic.", 20, 70 );
30
31       // set font to SansSerif (Helvetica), plain, 14pt and draw a string
32       g.setFont( new Font( "SansSerif", Font.PLAIN, 14 ) );
33       g.drawString( "SansSerif 14 point plain.", 20, 90 );
34
35       // set font to Serif (Times), bold/italic, 18pt and dra
36       g.setColor( Color.RED );
37       g.setFont( new Font( "Serif", Font.BOLD + Font.ITALIC, 18 ) );
38       g.drawString( g.getFont().getName() + " " + g.getFont().getSize() +
39          " point bold italic.", 20, 110 );
40
41    } // end method paint
42
43    // execute application
44    public static void main( String args[] )
45    {
46       Fonts application = new Fonts();
47       application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
48    }
49
50 } // end class Fonts
```

Set font to SansSerif 14-point plain

Set font to Serif 18-point bold italic



4.4. Font Control


```java
package Dr_Josbert_2D;

import java.awt.*;        // Provides classes for graphics (Graphics, Font, Color, etc.)
import javax.swing.*;     // Provides Swing components like JFrame

// Fonts class extends JFrame to create a window
public class Fonts extends JFrame {

   // Constructor: sets the window title and size
   public Fonts() {
      super("Using fonts");   // Set the title of the window

      setSize(420, 125);      // Set window width and height
      setVisible(true);       // Make the window visible
   }

   // paint method is used to draw on the window
   public void paint(Graphics g) {

      super.paint(g); // Calls the parent class paint method (important to avoid issues)

      // Set font to Serif (like Times New Roman), bold style, size 12
      g.setFont(new Font("Serif", Font.BOLD, 12));
      g.drawString("Serif 12 point bold.", 20, 50); // Draw text at (x=20, y=50)

      // Set font to Monospaced (like Courier), italic style, size 24
      g.setFont(new Font("Monospaced", Font.ITALIC, 24));
      g.drawString("Monospaced 24 point italic.", 20, 70);

      // Set font to SansSerif (like Helvetica), plain style, size 14
      g.setFont(new Font("SansSerif", Font.PLAIN, 14));
      g.drawString("SansSerif 14 point plain.", 20, 90);

      // Change text color to red
      g.setColor(Color.RED);
```

4.4. Font Control

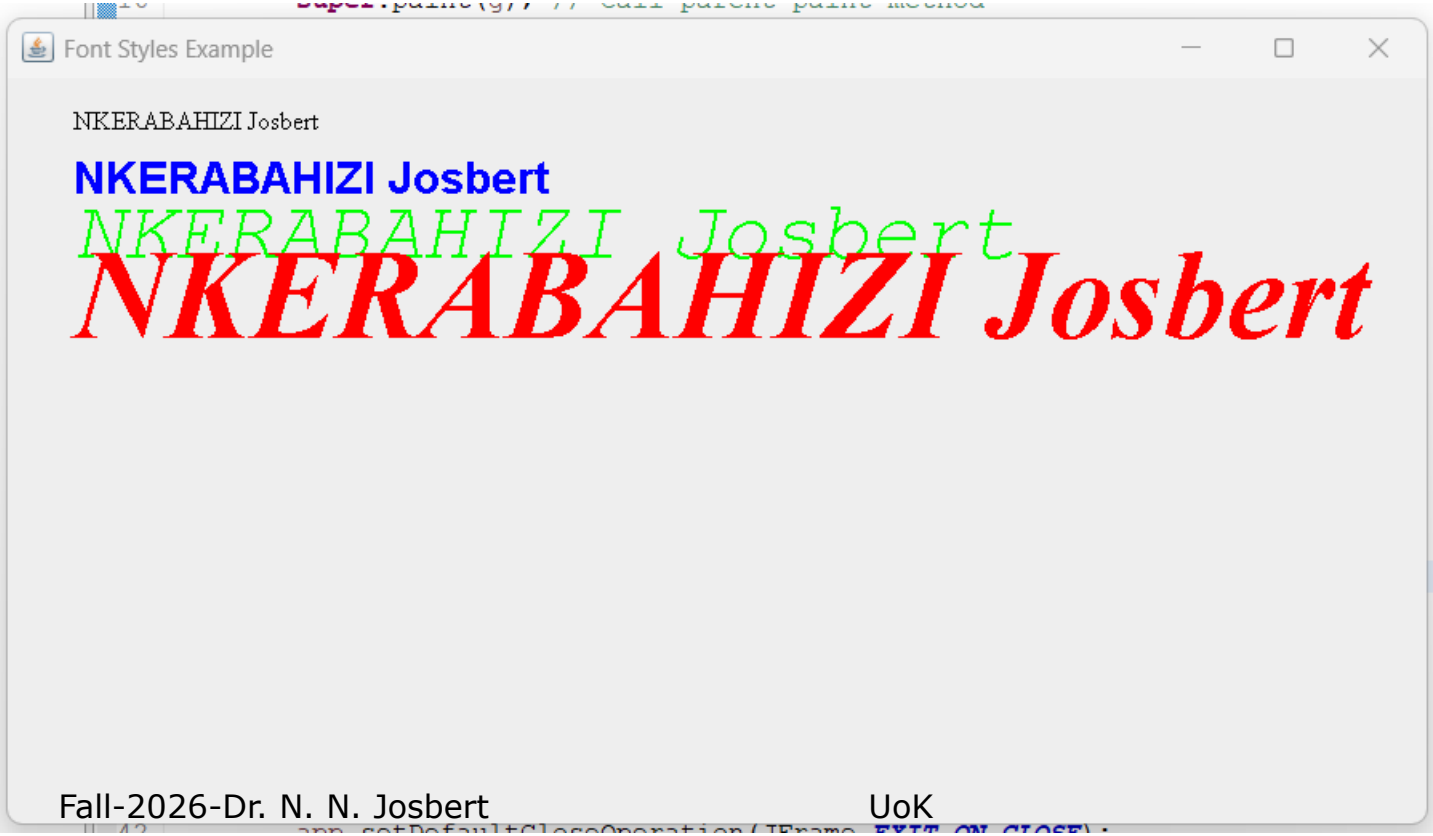

```
37        // Set font to Serif with both bold and italic styles, size 18
38        g.setFont(new Font("Serif Josbert", Font.BOLD + Font.ITALIC, 18));
39
40        // Display font name and size dynamically
41        g.drawString(
42           g.getFont().getName() + " " +
43           g.getFont().getSize() +
44           " point bold italic.",
45           20, 110
46        );
47
48     } // end method paint
49
50     // Main method: entry point of the program
51     public static void main(String args[]) {
52
53        Fonts application = new Fonts(); // Create an object of Fonts (opens window)
54
55        // Ensure program exits when window is closed
56        application.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
57     }
58
59  } // end class Fonts
60
```

4.4. Font Control


```java
package Dr_Josbert_2D;
import java.awt.*;
import javax.swing.*;

public class SimpleFonts extends JFrame {

    // Constructor to set up the window
    public SimpleFonts() {
        setTitle("Font Styles Example"); // Window title
        setSize(500, 300);               // Window size
        setVisible(true);                // Make window visible
    }

    // Method to draw text on the window
    public void paint(Graphics g) {
        super.paint(g); // Call parent paint method

        // Plain text (black, small size)
        g.setFont(new Font("Serif", Font.PLAIN, 16));
        g.setColor(Color.BLACK);
        g.drawString("NKERABAHIZI Josbert", 50, 70);

        // Bold text (blue, medium size)
        g.setFont(new Font("SansSerif", Font.BOLD, 28));
        g.setColor(Color.BLUE);
        g.drawString("NKERABAHIZI Josbert", 50, 110);

        // Italic text (green, larger size)
        g.setFont(new Font("Monospaced", Font.ITALIC, 52));
        g.setColor(Color.GREEN);
        g.drawString("NKERABAHIZI Josbert", 50, 150);

        // Bold + Italic text (red, biggest size)
        g.setFont(new Font("Serif", Font.BOLD + Font.ITALIC, 80));
        g.setColor(Color.RED);
        g.drawString("NKERABAHIZI Josbert", 50, 200);
```

4.4. Font Control


```
38
39      // Main method
40      public static void main(String[] args) {
41          SimpleFonts app = new SimpleFonts();
42          app.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
43      }
44  }
45
```



4.5. Drawing Lines, Rectangles and Ova

- **Class Graphics:**
  - Provides methods for drawing lines, rectangles and ovals.
    - All drawing methods require parameters x and y.
    - Almost all drawing methods require parameters width and height.

4.5. Drawing Lines, Rectangles and Ovals

`Graphics` methods that draw lines, rectangles and ovals

Method	Description
`public void drawLine( int x1, int y1, int x2, int y2 )`	
	Draws a line between the point (`x1`, `y1`) and the point (`x2`, `y2`).
`public void drawRect( int x, int y, int width, int height )`	
	Draws a rectangle of the specified `width` and `height`. The top-left corner of the rectangle has the coordinates (`x`, `y`).
`public void fillRect( int x, int y, int width, int height )`	
	Draws a solid rectangle with the specified `width` and `height`. The top-left corner of the rectangle has the coordinate (`x`, `y`).
`public void clearRect( int x, int y, int width, int height )`	
	Draws a solid rectangle with the specified `width` and `height` in the current background color. The top-left corner of the rectangle has the coordinate (`x`, `y`).
`public void drawRoundRect( int x, int y, int width, int height, int arcWidth, int arcHeight )`	
	Draws a rectangle with rounded corners in the current color with the specified `width` and `height`. The `arcWidth` and `arcHeight` determine the rounding of the corners (see Fig. 12.15).
`public void fillRoundRect( int x, int y, int width, int height, int arcWidth, int arcHeight )`	
	Draws a solid rectangle with rounded corners in the current color with the specified `width` and `height`. The `arcWidth` and `arcHeight` determine the rounding of the corners (see Fig. 12.15).

4.5. Drawing Lines, Rectangles and Ova

`Graphics` methods that draw lines, rectangles and ovals

Method	Description
`public void draw3DRect( int x, int y, int width, int height, boolean b )`	
	Draws a three-dimensional rectangle in the current color with the specified `width` and `height`. The top-left corner of the rectangle has the coordinates (`x`, `y`). The rectangle appears raised when `b` is true and lowered when `b` is false.
`public void fill3DRect( int x, int y, int width, int height, boolean b )`	
	Draws a filled three-dimensional rectangle in the current color with the specified `width` and `height`. The top-left corner of the rectangle has the coordinates (`x`, `y`). The rectangle appears raised when `b` is true and lowered when `b` is false.
`public void drawOval( int x, int y, int width, int height )`	
	Draws an oval in the current color with the specified `width` and `height`. The bounding rectangle's top-left corner is at the coordinates (`x`, `y`). The oval touches all four sides of the bounding rectangle at the center of each side (see Fig. 12.16).
`public void fillOval( int x, int y, int width, int height )`	
	Draws a filled oval in the current color with the specified `width` and `height`. The bounding rectangle's top-left corner is at the coordinates (`x`, `y`). The oval touches all four sides of the bounding rectangle at the center of each side (see Fig. 12.16).

4.5. Drawing Lines, Rectangles and Ova

```
2  // Drawing lines, rectangles and ovals.
3  import java.awt.*;
4  import javax.swing.*;
5
6  public class LinesRectsOvals extends JFrame {
7
8     // set window's title bar String and dimensions
9     public LinesRectsOvals()
10    {
11       super( "Drawing lines, rectangles and ovals" );
12
13       setSize( 400, 165 );
14       setVisible( true );
15    }
16
17    // display various lines, rectangles and ovals
18    public void paint( Graphics g )
19    {
20       super.paint( g );  // call superclass's paint method
21
22       g.setColor( Color.RED );
23       g.drawLine( 5, 30, 350, 30 );
24
25       g.setColor( Color.BLUE );
26       g.drawRect( 5, 40, 90, 55 );
27       g.fillRect( 100, 40, 90, 55 );
```

4.5. Drawing Lines, Rectangles and Ova


```
28
29        g.setColor( Color.CYAN );
30        g.fillRoundRect( 195, 40, 90, 55, 50, 50 );
31        g.drawRoundRect( 290, 40, 90, 55, 20, 20 );
32
33        g.setColor( Color.YELLOW );
34        g.draw3DRect( 5, 100, 90, 55, true );
35        g.fill3DRect( 100, 100, 90, 55, false );
36
37        g.setColor( Color.MAGENTA );
38        g.drawOval( 195, 100, 90, 55 );
39        g.fillOval( 290, 100, 90, 55 );
40
41     } // end method paint
42
43     // execute application
44     public static void main( String args[] )
45     {
46        LinesRectsOvals application = new LinesRectsOvals();
47        application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
48     }
49
50  } // end class LinesRectsOvals
```

Draw filled rounded rectangle

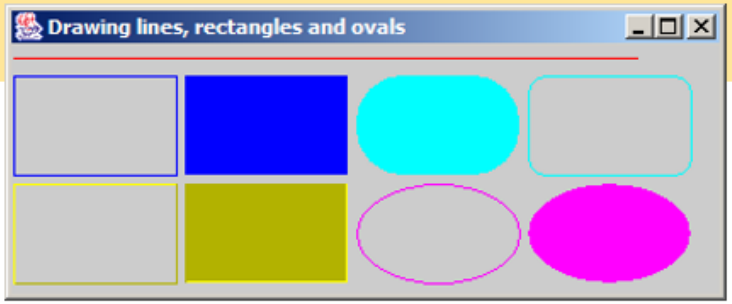
Draw (non-filled) rounded rectangle

Draw 3D rectangle

Draw filled 3D rectangle

Draw oval

Draw filled oval



Outline

LinesRectsOvals

Line 30

Line 31

Line 34

Line 35

Line 38

Line 39

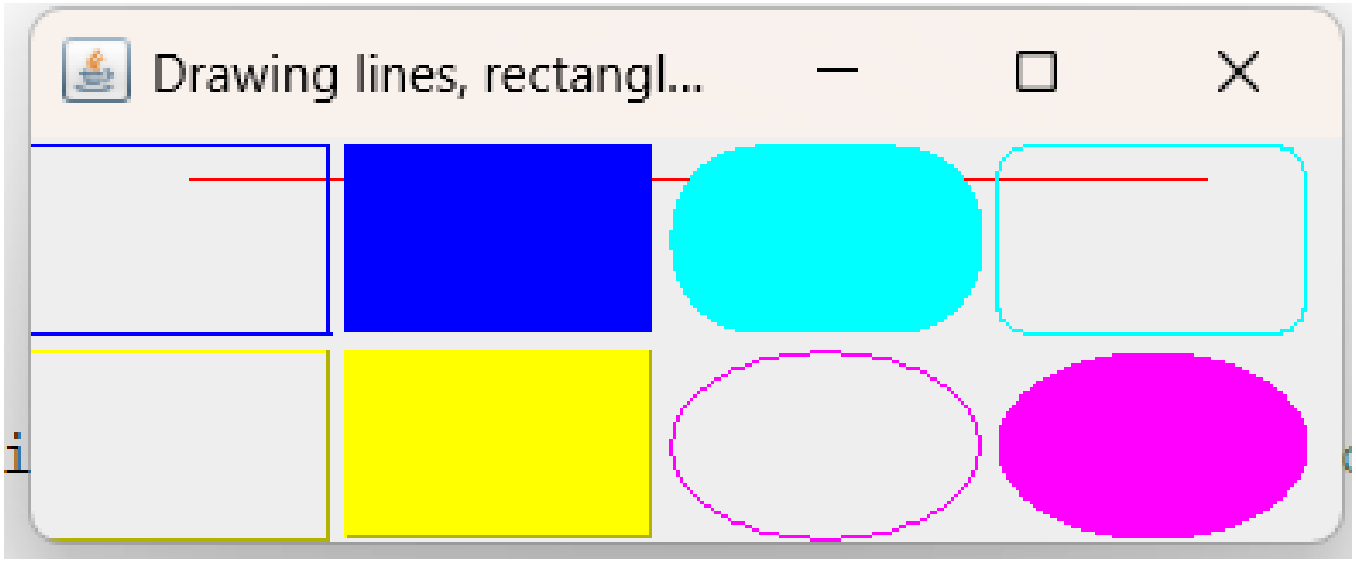
4.5. Drawing Lines, Rectangles and Ova


```java
package Dr_Josbert_2D;

//Drawing lines, rectangles and ovals.
import java.awt.*;        // Provides classes for graphics (Graphics, Color, etc.)
import javax.swing.*;     // Provides GUI components like JFrame

//Class that extends JFrame to create a window
public class LinesRectsOvals extends JFrame {

// Constructor: sets the window title and size
public LinesRectsOvals() {
   super("Drawing lines, rectangles and ovals"); // Set title of window

   setSize(400, 165);    // Set window width and height
   setVisible(true);     // Make window visible on screen
}

// paint method is used to draw shapes on the window
public void paint(Graphics g) {

   super.paint(g);  // Call superclass paint method (clears screen and prepares drawing)

   // ---------------- DRAWING A LINE ----------------
   g.setColor(Color.RED);              // Set drawing color to red
   g.drawLine(55, 50, 350, 50);        // Draw a horizontal line from (15,50) to (350,50)

   // ---------------- RECTANGLES ----------------
   g.setColor(Color.BLUE);             // Change color to blue

   g.drawRect(5, 40, 90, 55);          // Draw an outline rectangle
   // Parameters: x, y, width, height

   g.fillRect(100, 40, 90, 55);        // Draw a filled rectangle (solid color)

```

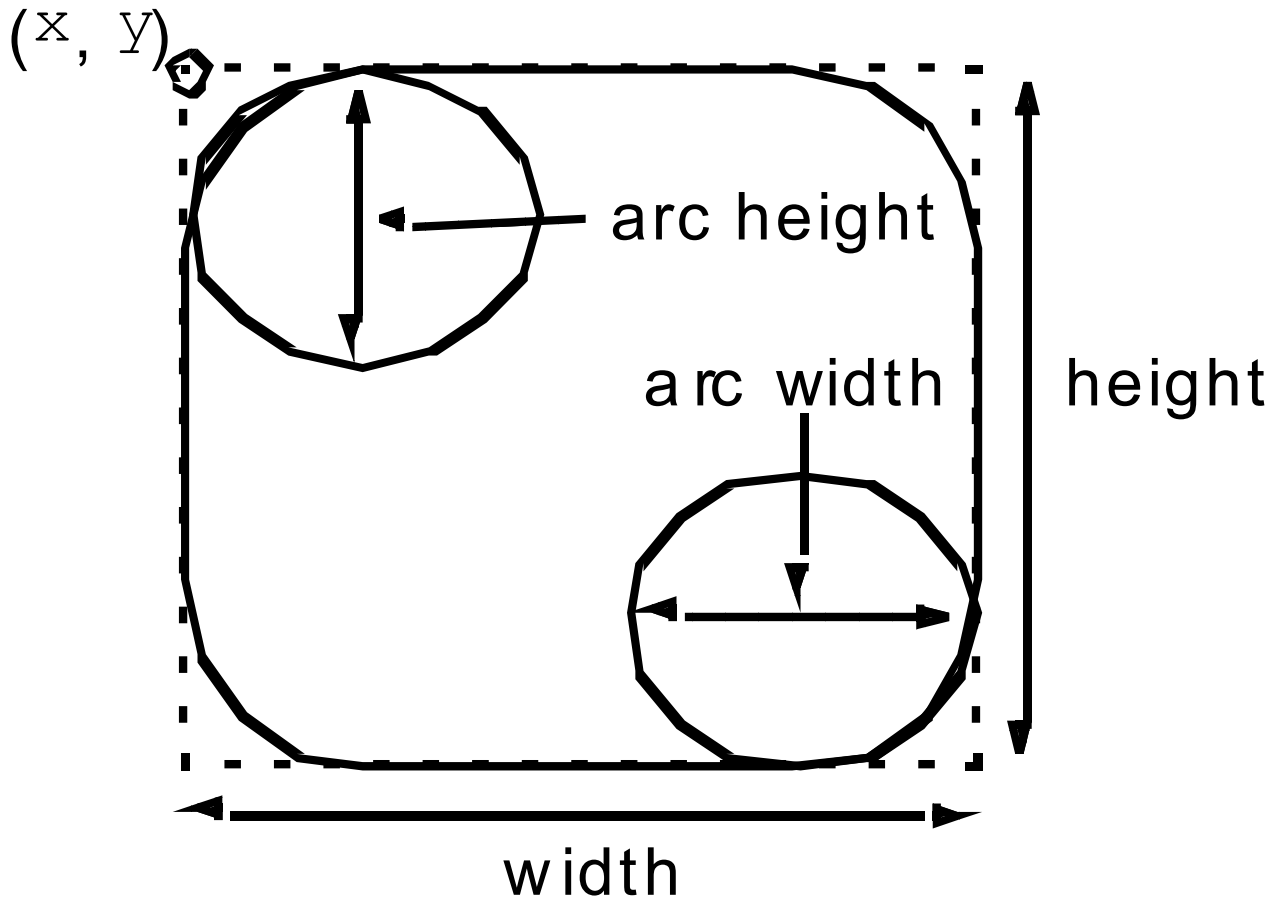
4.5. Drawing Lines, Rectangles and Ova


```
35      // ---------------- ROUNDED RECTANGLES ----------------
36      g.setColor(Color.CYAN);             // Change color to cyan
37
38      g.fillRoundRect(195, 40, 90, 55, 50, 50);
39      // Filled rounded rectangle (last two values = arc width & height)
40
41      g.drawRoundRect(290, 40, 90, 55, 20, 20);
42      // Outline rounded rectangle
43
44      // ---------------- 3D RECTANGLES ----------------
45      g.setColor(Color.YELLOW);           // Change color to yellow
46
47      g.draw3DRect(5, 100, 90, 55, true);
48      // Draw 3D rectangle (true = raised effect)
49
50      g.fill3DRect(100, 100, 90, 55, true);
51      // Filled 3D rectangle (false = sunken effect)
52
53      // ---------------- OVALS ----------------
54      g.setColor(Color.MAGENTA);          // Change color to magenta
55
56      g.drawOval(195, 100, 90, 55);       // Draw outline oval
57
58      g.fillOval(290, 100, 90, 55);       // Draw filled oval
59
60  } // end method paint
61
62  // Main method: program entry point
63  public static void main(String args[]) {
64
65      LinesRectsOvals application = new LinesRectsOvals(); // Create window object
66
67      // Close the program when the window is closed
68      application.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
69  }
70
71  } // end class LinesRectsOvals
```

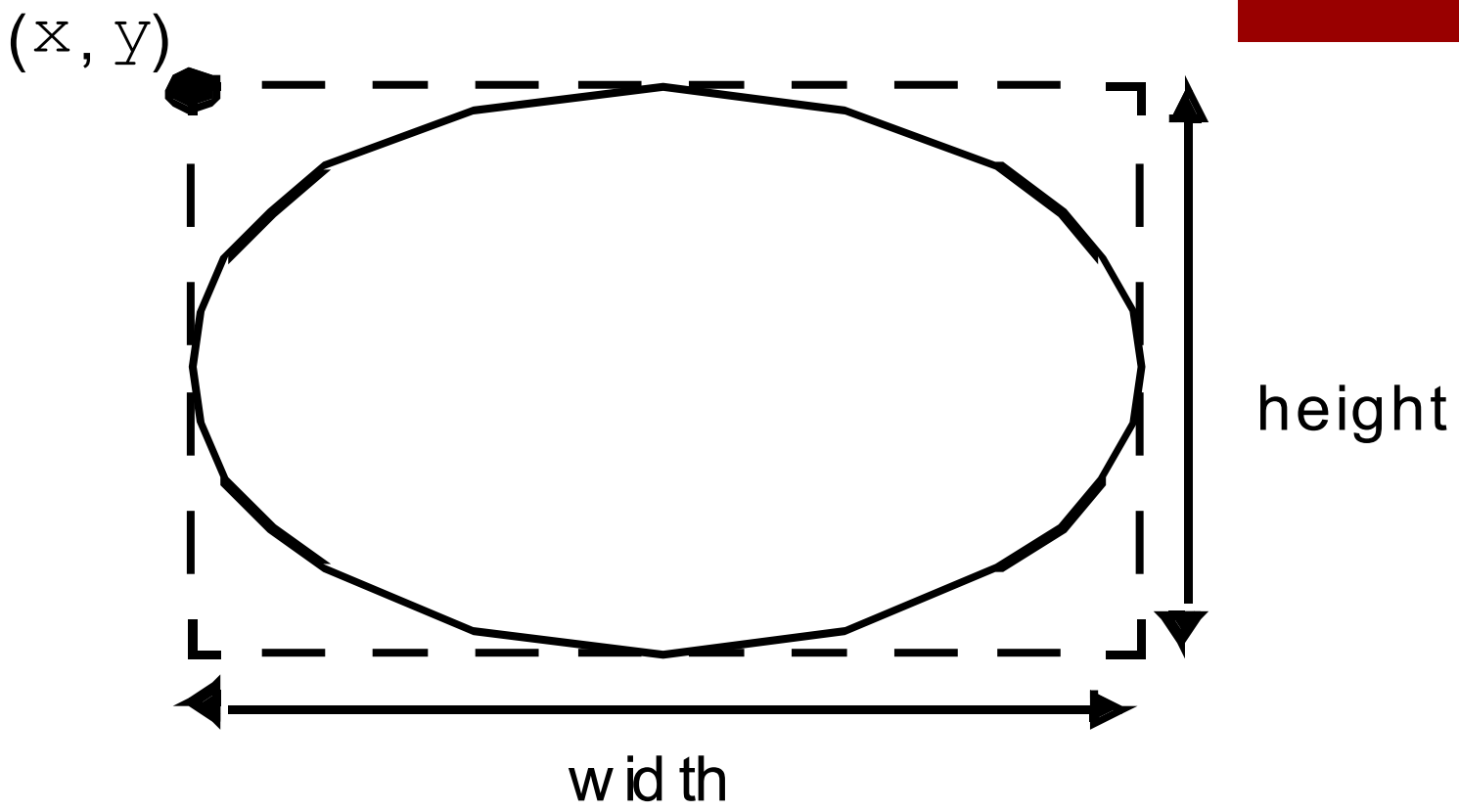
4.5. Drawing Lines, Rectangles and Ova



4.6. Arc width and arc height for rounded rectangles

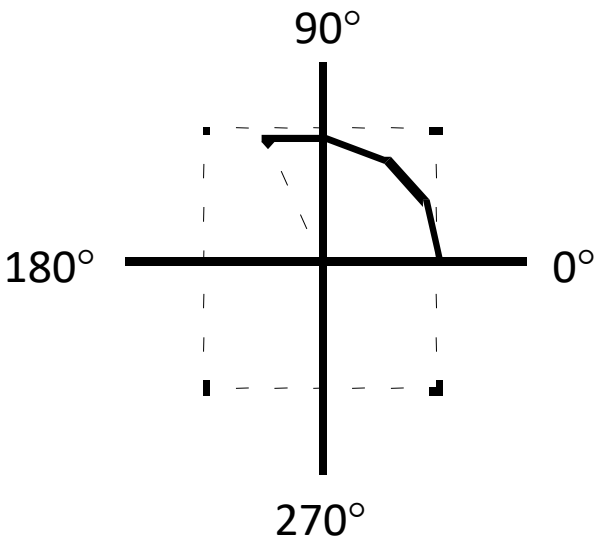


4.6. Oval bounded by a rectangle

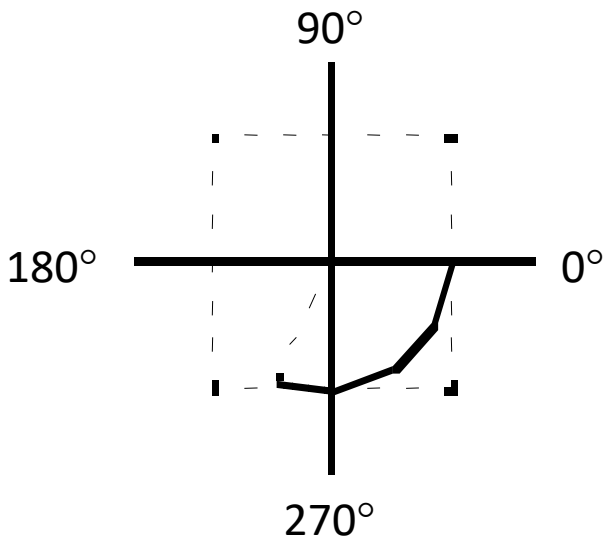


4.7. Positive and negative arc angles

Positive angles

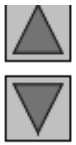


Negative angles



4.7. Graphics methods for drawing arcs

Method	Description
`public void drawArc( int x, int y, int width, int height, int startAngle, int arcAngle )`	
	Draws an arc relative to the bounding rectangle's top-left coordinates *(x, y)* with the specified `width` and `height`. The arc segment is drawn starting at `startAngle` and sweeps `arcAngle` degrees.
`public void fillArc( int x, int y, int width, int height, int startAngle, int arcAngle )`	
	Draws a solid arc (i.e., a sector) relative to the bounding rectangle's top-left coordinates *(x, y)* with the specified `width` and `height`. The arc segment is drawn starting at `startAngle` and sweeps `arcAngle` degrees.



Outline

DrawArcs.java

Lines 24-26

```
1  // Fig. 12.19: DrawArcs.java
2  // Drawing arcs.
3  import java.awt.*;
4  import javax.swing.*;
5
6  public class DrawArcs extends JFrame {
7
8     // set window's title bar String and dimensions
9     public DrawArcs()
10    {
11       super( "Drawing Arcs" );
12
13       setSize( 300, 170 );
14       setVisible( true );
15    }
16
17    // draw rectangles and arcs
18    public void paint( Graphics g )
19    {
20       super.paint( g );  // call superclass's paint method
21
22       // start at 0 and sweep 360 degrees
23       g.setColor( Color.YELLOW );
24       g.drawRect( 15, 35, 80, 80 );
25       g.setColor( Color.BLACK );
26       g.drawArc( 15, 35, 80, 80, 0, 360 );
```

Draw first arc that sweeps 360 degrees and is contained in rectangle

Outline

Arcs.java

Lines 30-32

Lines 36-38

Line 44

```
27
28        // start at 0 and sweep 110 degrees
29        g.setColor( Color.YELLOW );
30        g.drawRect( 100, 35, 80, 80 );
31        g.setColor( Color.BLACK );
32        g.drawArc( 100, 35, 80, 80, 0, 110 );
33
34        // start at 0 and sweep -270 degrees
35        g.setColor( Color.YELLOW );
36        g.drawRect( 185, 35, 80, 80 );
37        g.setColor( Color.BLACK );
38        g.drawArc( 185, 35, 80, 80, 0, -270 );
39
40        // start at 0 and sweep 360 degrees
41        g.fillArc( 15, 120, 80, 40, 0, 360 );
42
43        // start at 270 and sweep -90 degrees
44        g.fillArc( 100, 120, 80, 40, 270, -90 );
45
46        // start at 0 and sweep -270 degrees
47        g.fillArc( 185, 120, 80, 40, 0, -270 );
48
49     } // end method paint
50
```

Draw second arc that sweeps 110 degrees and is contained in rectangle

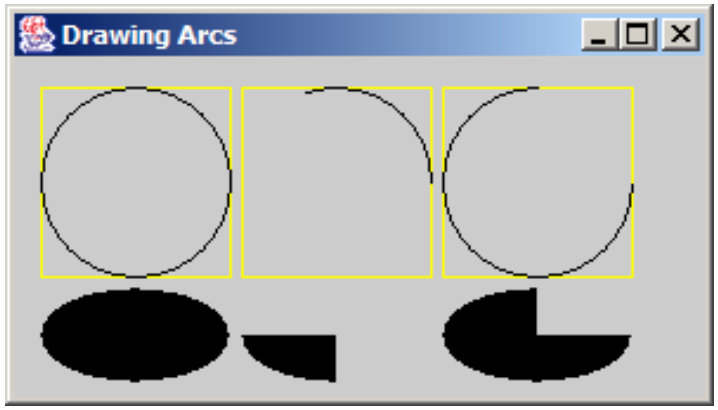
Draw third arc that sweeps -270 degrees and is contained in rectangle

Draw fourth arc that is filled, has starting angle 0 and sweeps 360 degrees

Draw fifth arc that is filled, has starting angle 270 and sweeps -90 degrees

Draw sixth arc that is filled, has starting angle 0 and sweeps -270 degrees

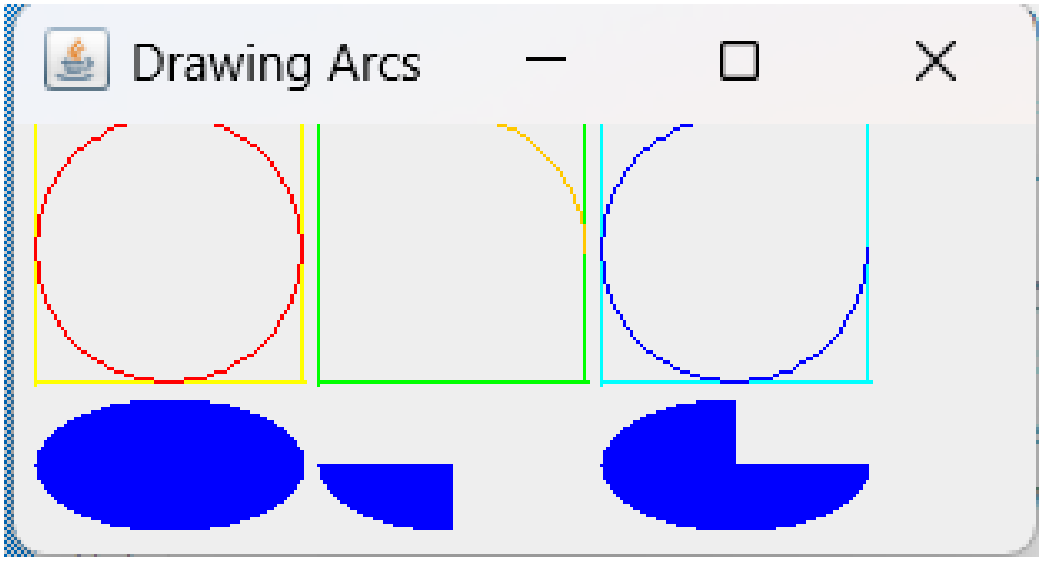
```
51     // execute application
52     public static void main( String args[] )
53     {
54        DrawArcs application = new DrawArcs();
55        application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
56     }
57
58  } // end class DrawArcs
```



```java
package Dr_Josbert_2D;

//---------------- IMPORTING LIBRARIES ----------------

//Import AWT package for drawing (Graphics, Color, etc.)
import java.awt.*;

//Import Swing package for GUI components (JFrame)
import javax.swing.*;

//---------------- MAIN CLASS ----------------
public class DrawArcs extends JFrame {

 // ---------------- CONSTRUCTOR ----------------
 // This sets the window title and size
 public DrawArcs() {

     super("Drawing Arcs"); // Set the title of the window

     setSize(300, 170);     // Set window width = 300, height = 170

     setVisible(true);      // Make the window visible on screen
 }

 // ---------------- PAINT METHOD ----------------
 // This method is used to draw shapes on the window
 public void paint(Graphics g) {

     super.paint(g);  // VERY IMPORTANT: clears screen before drawing

     // ============================================================
     // FIRST ARC (FULL CIRCLE - 360 degrees)
     // ============================================================

     g.setColor(Color.YELLOW);          // Set color to YELLOW
     g.drawRect(15, 35, 80, 80);        // Draw square boundary (guide)
```

```
37
38      g.setColor(Color.RED);            // Set color to BLACK for arc
39      g.drawArc(15, 35, 80, 80, 0, 360);
40      // Draw arc:
41      // (x=15, y=35) → starting position
42      // width=80, height=80 → size of oval
43      // start angle = 0 degrees (right side)
44      // arc angle = 360 → full circle
45
46      // ============================================================
47      // SECOND ARC (PARTIAL ARC - 110 degrees)
48      // ============================================================
49
50      g.setColor(Color.GREEN);
51      g.drawRect(100, 35, 80, 80);       // Draw boundary rectangle
52
53      g.setColor(Color.ORANGE);
54      g.drawArc(100, 35, 80, 80, 0, 110);
55      // Draw arc starting at 0 degrees
56      // Sweep 110 degrees (small curved shape)
57
58      // ============================================================
59      // THIRD ARC (NEGATIVE DIRECTION - CLOCKWISE)
60      // ============================================================
61
62      g.setColor(Color.CYAN);
63      g.drawRect(185, 35, 80, 80);       // Draw boundary
64
65      g.setColor(Color.BLUE);
66      g.drawArc(185, 35, 80, 80, 0, -270);
67      // Negative value = draws arc in clockwise direction
68      // -270 means large arc going opposite direction
69
70      // ============================================================
71      // FILLED ARCS (SOLID SHAPES)
72      // ============================================================
```

```
73
74      // FULL FILLED ARC (looks like filled oval)
75      g.fillArc(15, 120, 80, 40, 0, 360);
76      // width=80, height=40 → oval shape
77      // 360 degrees = full filled oval
78
79      // PARTIAL FILLED ARC (quarter shape)
80      g.fillArc(100, 120, 80, 40, 270, -90);
81      // Start at 270 degrees (bottom)
82      // Sweep -90 → clockwise quarter arc
83
84      // LARGE FILLED ARC (almost full circle)
85      g.fillArc(185, 120, 80, 40, 0, -270);
86      // Large filled arc covering most of the shape
87
88  } // end method paint
89
90  // ---------------- MAIN METHOD ----------------
91  // Program starts execution here
92  public static void main(String args[]) {
93
94      DrawArcs application = new DrawArcs(); // Create window object
95
96      // Close program when user clicks "X"
97      application.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
98  }
99
100 } // end class DrawArcs
101
```


Drawing Arcs

4.8. Drawing Polygons and Polylines

- Class Polygon
  - Polygons
    - ❖Multisided shapes
  - Polylines
    - ❖Series of connected points

4.8. Drawing Polygons and Polylines

Graphics methods for drawing polygons and class Polygon methods

Method	Description
`Graphics` methods for drawing polygons	
`public void drawPolygon( int xPoints[], int yPoints[], int points )`	
	Draws a polygon. The *x*-coordinate of each point is specified in the `xPoints` array and the *y*-coordinate of each point is specified in the `yPoints` array. The last argument specifies the number of `points`. This method draws a closed polygon. If the last point is different from the first point, the polygon is closed by a line that connects the last point to the first point.
`public void drawPolyline( int xPoints[], int yPoints[], int points )`	
	Draws a sequence of connected lines. The *x*-coordinate of each point is specified in the `xPoints` array and the *y*-coordinate of each point is specified in the `yPoints` array. The last argument specifies the number of `points`. If the last point is different from the first point, the polyline is not closed.
`public void drawPolygon( Polygon p )`	
	Draws the specified polygon.
`public void fillPolygon( int xPoints[], int yPoints[], int points )`	
	Draws a solid polygon. The *x*-coordinate of each point is specified in the `xPoints` array and the *y*-coordinate of each point is specified in the `yPoints` array. The last argument specifies the number of `points`. This method draws a closed polygon. If the last point is different from the first point, the polygon is closed by a line that connects the last point to the first point.
`public void fillPolygon( Polygon p )`	
	Draws the specified solid polygon. The polygon is closed.

4.8. Drawing Polygons and Polylines

Graphics methods for drawing polygons and class Polygon methods

Method	Description
`Polygon` constructors and methods	
`public Polygon()`	
	Constructs a new polygon object. The polygon does not contain any points.
`public Polygon( int xValues[], int yValues[], int numberOfPoints )`	
	Constructs a new polygon object. The polygon has `numberOfPoints` sides, with each point consisting of an *x*-coordinate from `xValues` and a *y*-coordinate from `yValues`.
`public void addPoint( int x, int y )`	
	Adds pairs of *x*- and *y*-coordinates to the `Polygon`.

```
2  // Drawing polygons.
3  import java.awt.*;
4  import javax.swing.*;
5
6  public class DrawPolygons extends JFrame {
7
8     // set window's title bar String and dimensions
9     public DrawPolygons()
10    {
11       super( "Drawing Polygons" );
12
13       setSize( 275, 230 );
14       setVisible( true );
15    }
16
17    // draw polygons and polylines
18    public void paint( Graphics g )
19    {
20       super.paint( g );  // call superclass's paint method
21
22       int xValues[] = { 20, 40, 50, 30, 20, 15 };
23       int yValues[] = { 50, 50, 60, 80, 80, 60 };
24       Polygon polygon1 = new Polygon( xValues, yValues, 6 );
25
26       g.drawPolygon( polygon1 );
27
```

DrawPolygons.java

Lines 22-23

Line 26

`int` arrays specifying `Polygon polygon1` points

Draw `polygon1` to screen

Outline

```
28        int xValues2[] = { 70, 90, 100, 80, 70, 65, 60 };
29        int yValues2[] = { 100, 100, 110, 110, 130, 110, 90 };
30
31        g.drawPolyline( xValues2, yValues2, 7 );
32
33        int xValues3[] = { 120, 140, 150, 190 };
34        int yValues3[] = { 40, 70, 80, 60 };
35
36        g.fillPolygon( xValues3, yValues3, 4 );
37
38        Polygon polygon2 = new Polygon();
39        polygon2.addPoint( 165, 135 );
40        polygon2.addPoint( 175, 150 );
41        polygon2.addPoint( 270, 200 );
42        polygon2.addPoint( 200, 220 );
43        polygon2.addPoint( 130, 180 );
44
45        g.fillPolygon( polygon2 );
46
47     } // end method paint
48
49     // execute application
50     public static void main( String args[] )
51     {
52        DrawPolygons application = new DrawPolygons();
53        application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
54     }
55
56  } // end class DrawPolygons
```

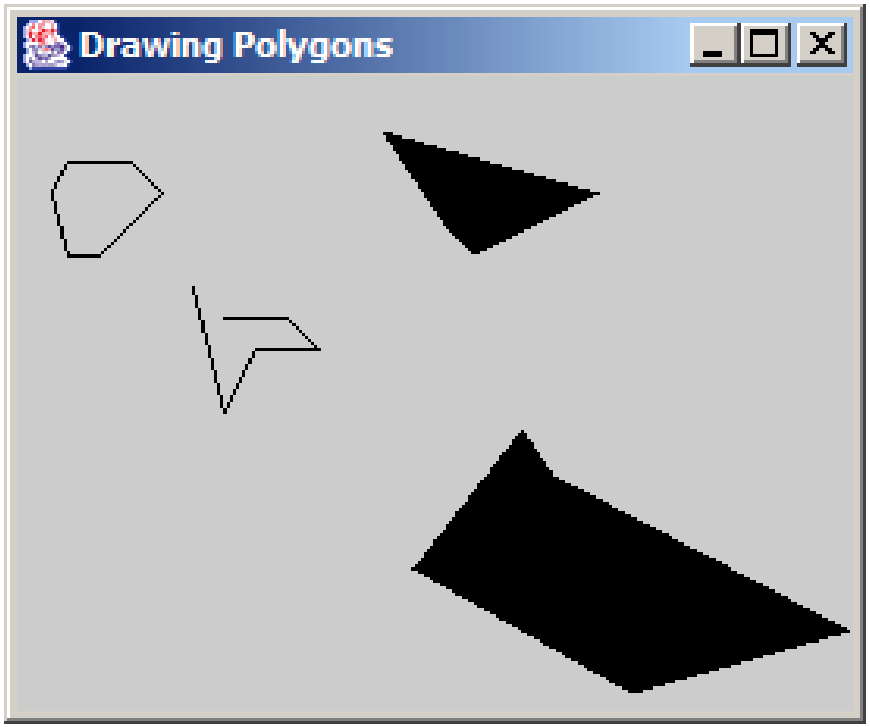
`int` arrays specifying `Polyline` points

Draw `Polyline` to screen

Lines 28-29

Specify points and draw (filled) `Polygon` to screen

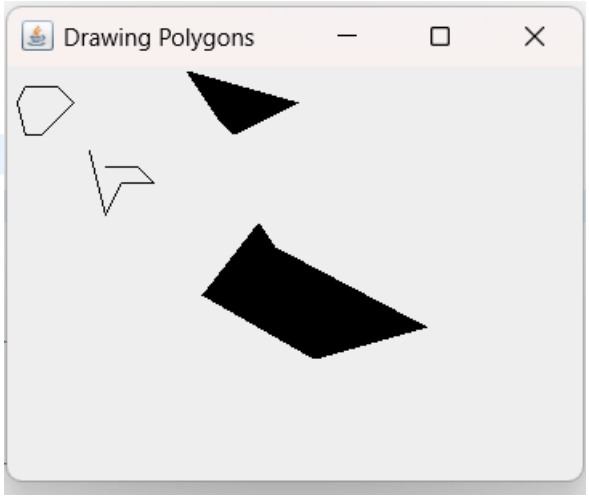
Method `addPoint` adds pairs of x-y coordinates to a `Polygon`


Drawing Polygons

```java
package Dr_Josbert_2D;

//---------------- IMPORT LIBRARIES ----------------

//AWT package → provides Graphics, Color, Polygon, etc.
import java.awt.*;

//Swing package → provides JFrame (window)
import javax.swing.*;

//---------------- MAIN CLASS ----------------
public class DrawPolygons extends JFrame {

 // ---------------- CONSTRUCTOR ----------------
 // This sets the window title and size
 public DrawPolygons() {

     super("Drawing Polygons"); // Title of the window

     setSize(275, 230);         // Window width = 275, height = 230

     setVisible(true);          // Make window visible
 }

 // ---------------- PAINT METHOD ----------------
 // This method is responsible for drawing shapes
 public void paint(Graphics g) {

     super.paint(g);  // IMPORTANT: clears screen before drawing

     // ============================================================
     // FIRST POLYGON (USING ARRAYS + Polygon OBJECT)
     // ============================================================
```

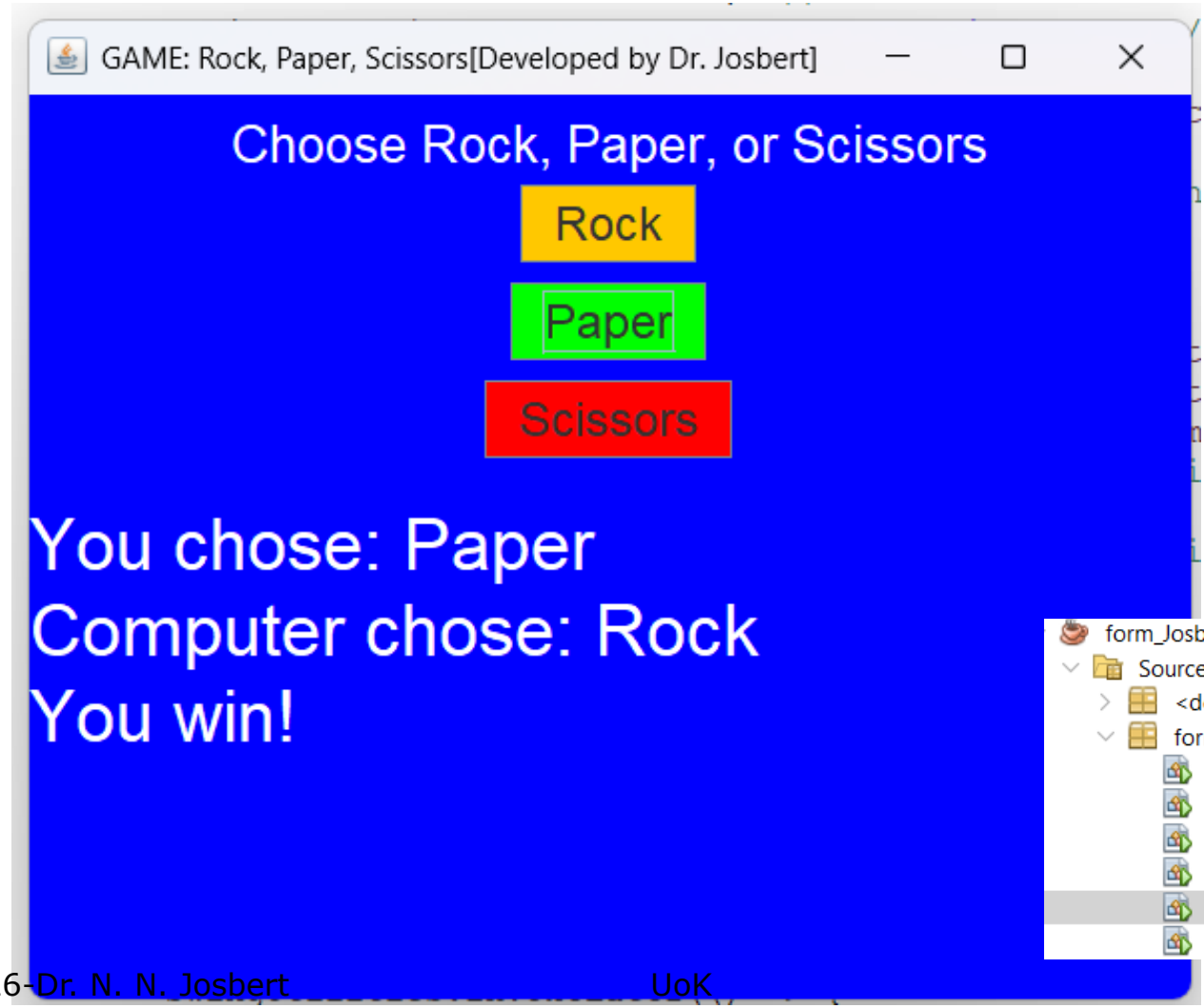
```
34
35      // X coordinates of points
36      int xValues[] = {20, 40, 50, 30, 20, 15};
37
38      // Y coordinates of points
39      int yValues[] = {50, 50, 60, 80, 80, 60};
40
41      // Create a Polygon object using coordinates
42      Polygon polygon1 = new Polygon(xValues, yValues, 6);
43      // 6 = number of points (must match array size)
44
45      g.drawPolygon(polygon1);
46      // Draws CLOSED shape by connecting all points
47      // Last point connects back to first automatically
48
49      // ============================================================
50      // SECOND SHAPE (POLYLINE - OPEN SHAPE)
51      // ============================================================
52
53      int xValues2[] = {70, 90, 100, 80, 70, 65, 60};
54      int yValues2[] = {100, 100, 110, 110, 130, 110, 90};
55
56      g.drawPolyline(xValues2, yValues2, 7);
57      // Draws OPEN shape (lines only)
58      // Does NOT connect last point to first point
59
60      // ============================================================
61      // THIRD SHAPE (FILLED POLYGON USING ARRAYS)
62      // ============================================================
63
64      int xValues3[] = {120, 140, 150, 190};
65      int yValues3[] = {40, 70, 80, 60};
66
67      g.fillPolygon(xValues3, yValues3, 4);
68      // Draws CLOSED and FILLED shape
69      // 4 = number of points
```

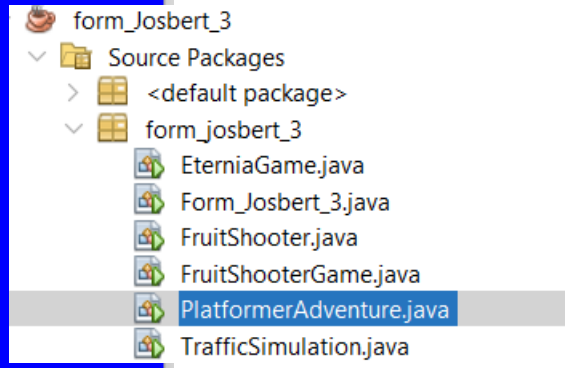
```
71      // ============================================================
72      // FOURTH SHAPE (POLYGON USING addPoint METHOD)
73      // ============================================================
74
75      Polygon polygon2 = new Polygon();
76      // Create empty polygon
77
78      // Add points one by one (x, y)
79      polygon2.addPoint(165, 135);
80      polygon2.addPoint(175, 150);
81      polygon2.addPoint(270, 200);
82      polygon2.addPoint(200, 220);
83      polygon2.addPoint(130, 180);
84
85      g.fillPolygon(polygon2);
86      // Draw and fill the polygon
87      // Automatically connects all points
88
89  } // end method paint
90
91  // --------------- MAIN METHOD ---------------
92  // Entry point of the program
93  public static void main(String args[]) {
94
95      DrawPolygons application = new DrawPolygons(); // Create window
96
97      // Close program when user clicks the close button
98      application.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
99  }
100
101 } // end class DrawPolygons
102
```



4. 9. Games Design and Animations

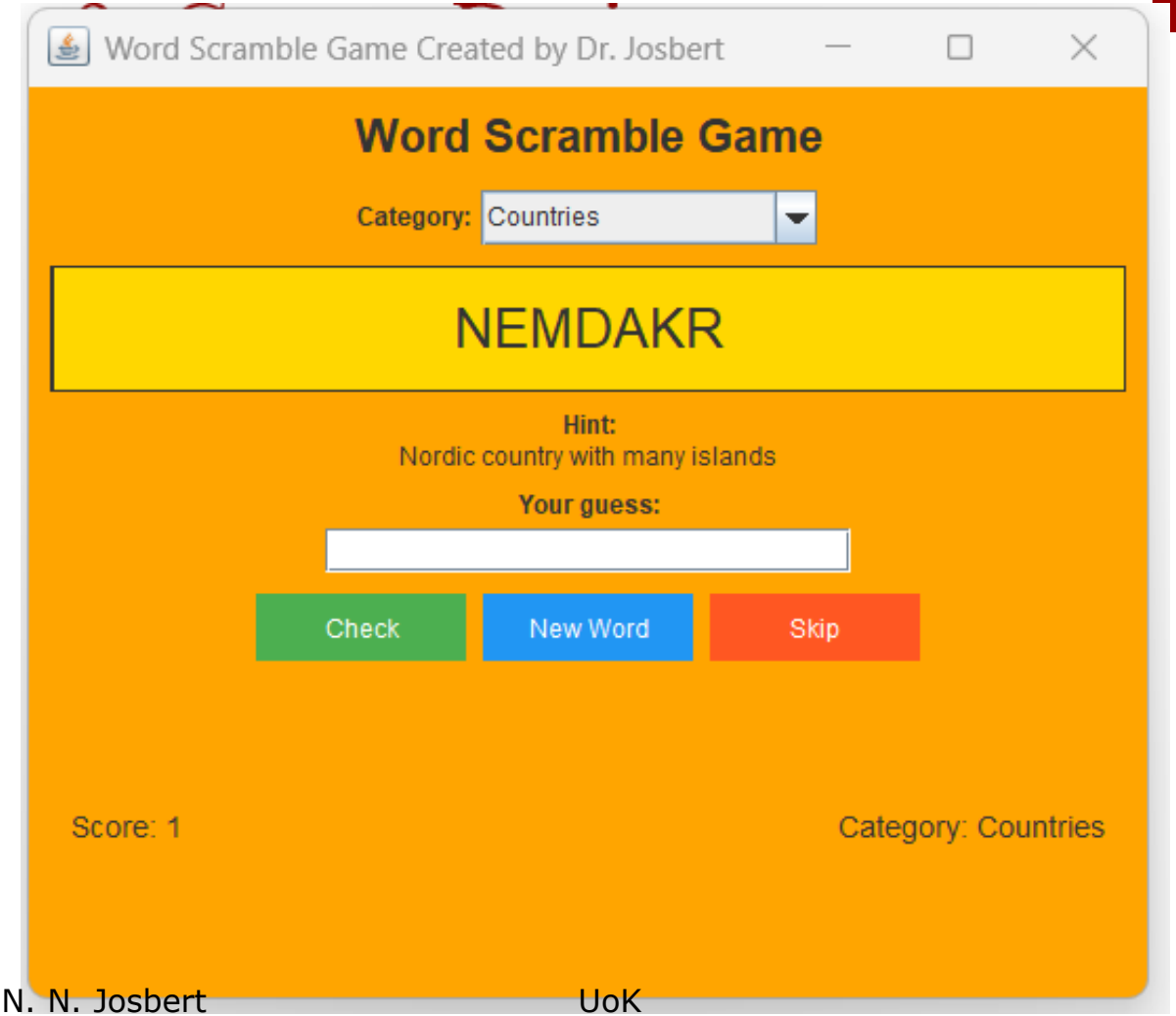
Rock, Paper, and Scissors





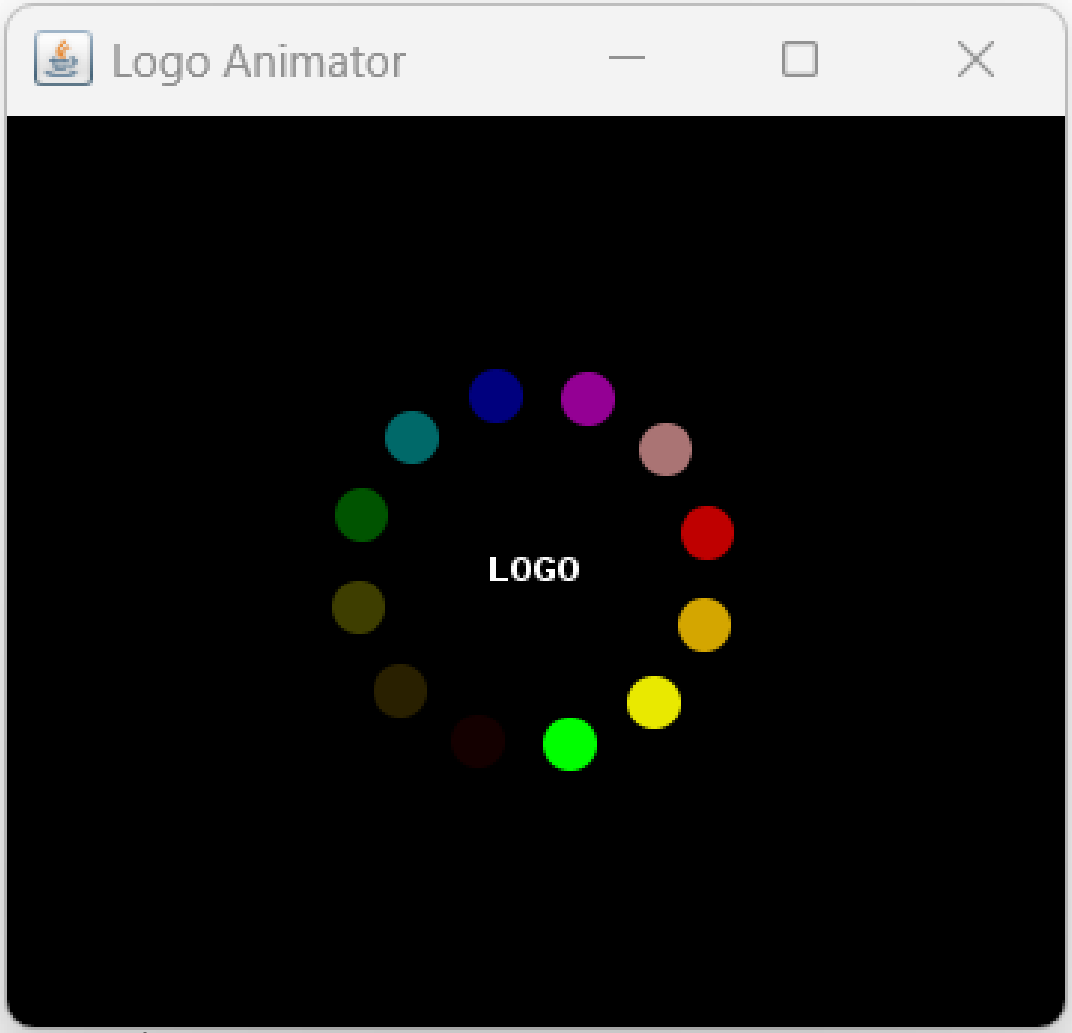
4. 9. Games Design

Word Scramble Game



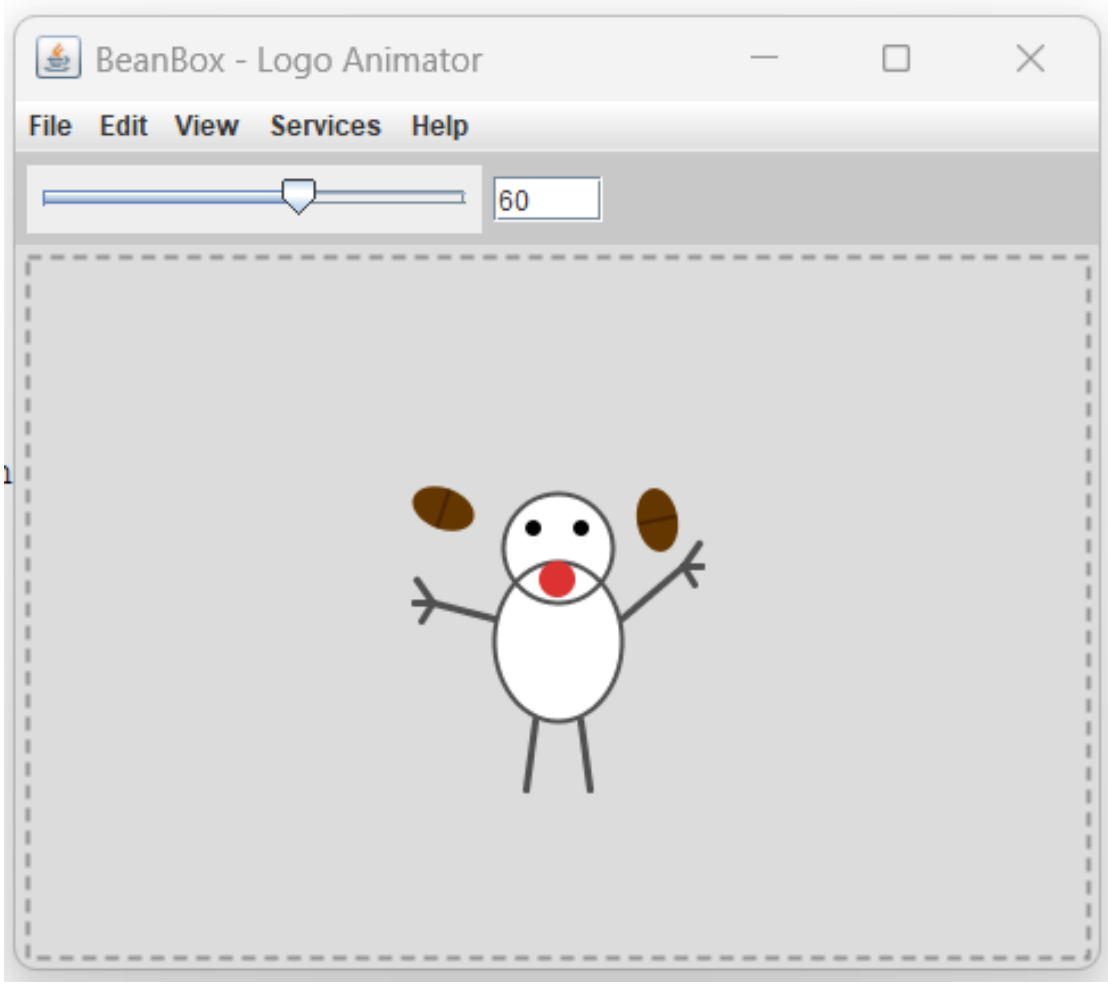
4. 9. Games Design

Logo Animation

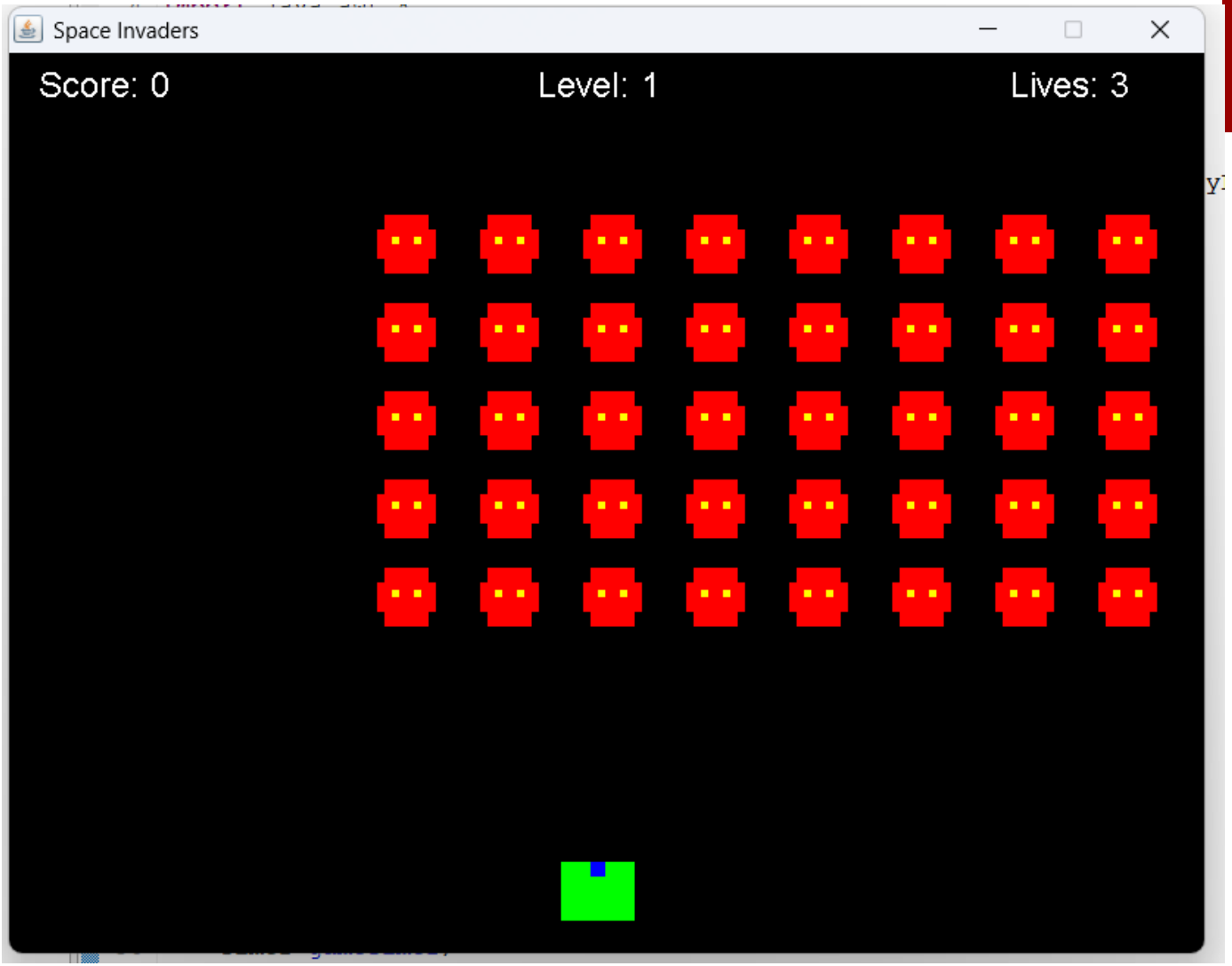


4. 9. Games Design

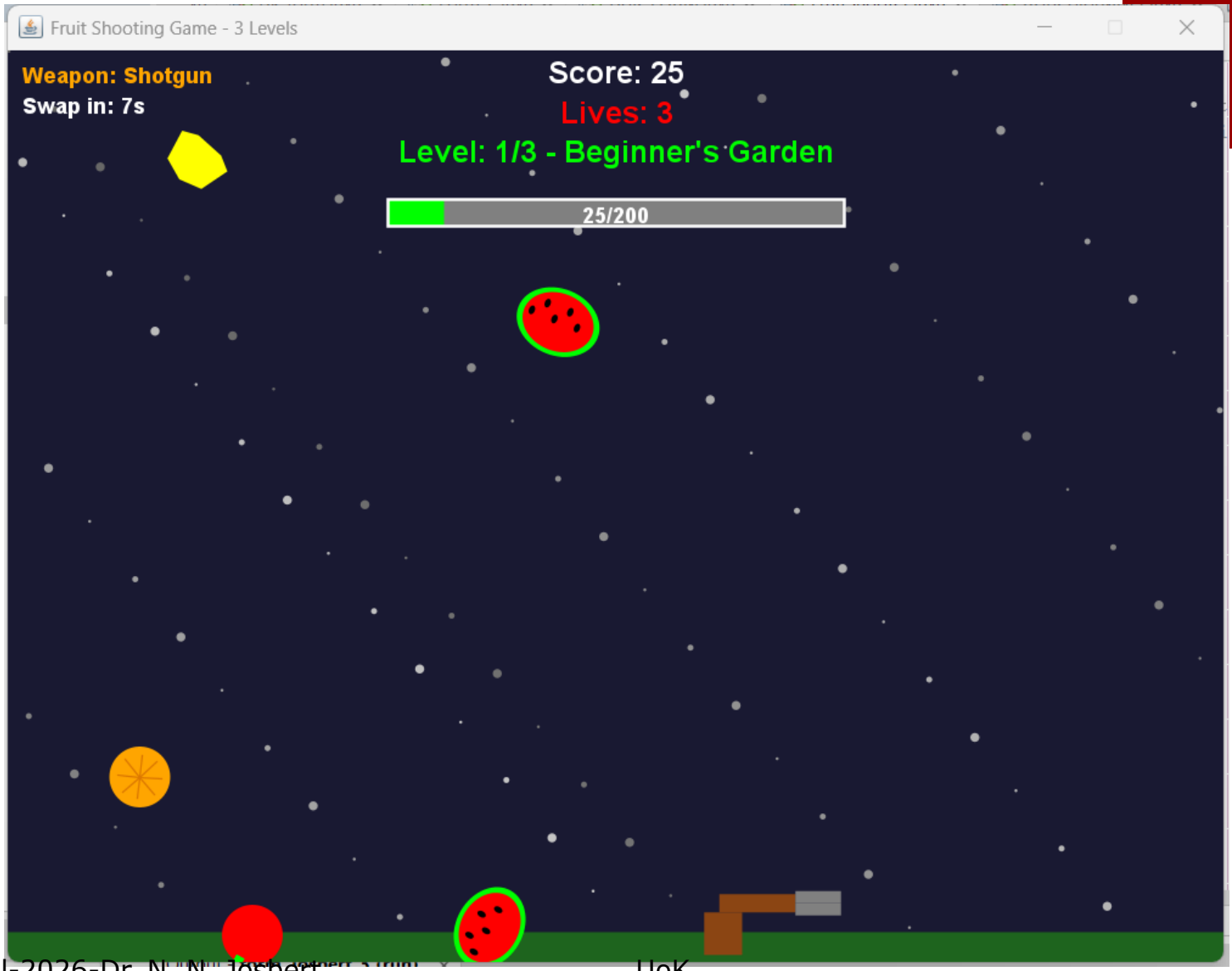
Gift Animation



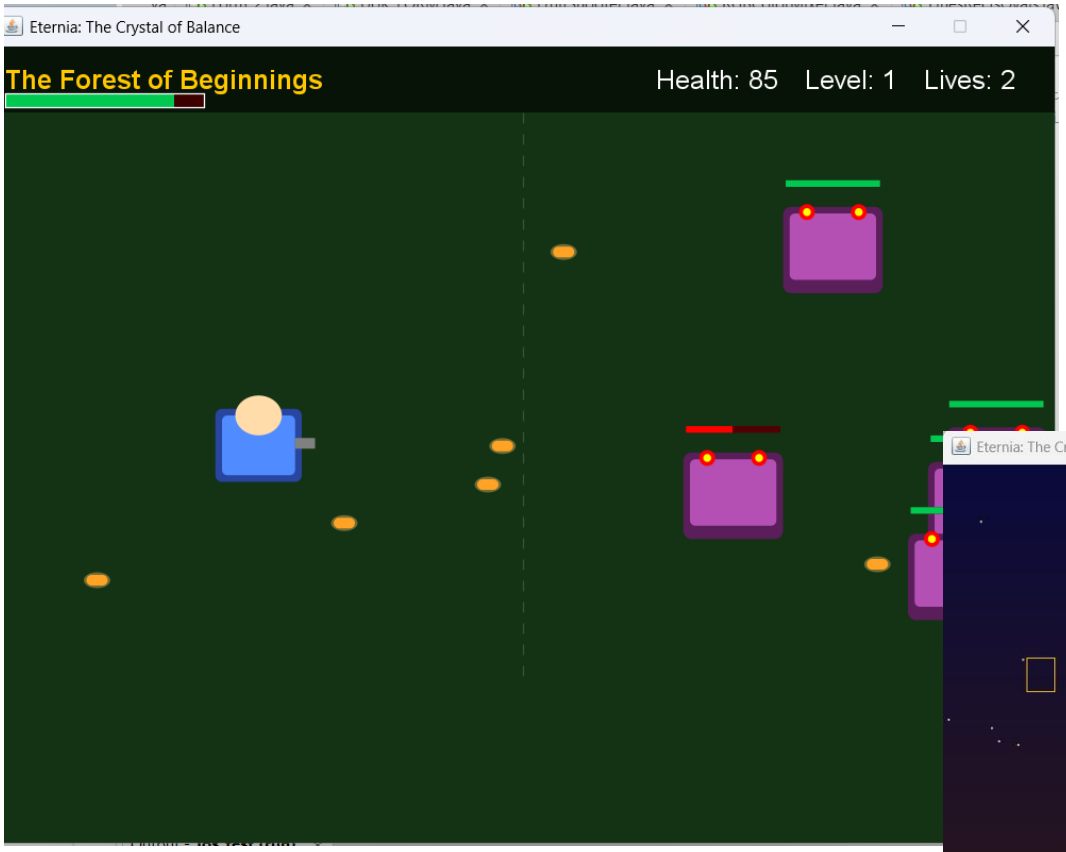
4. 9. Games Design

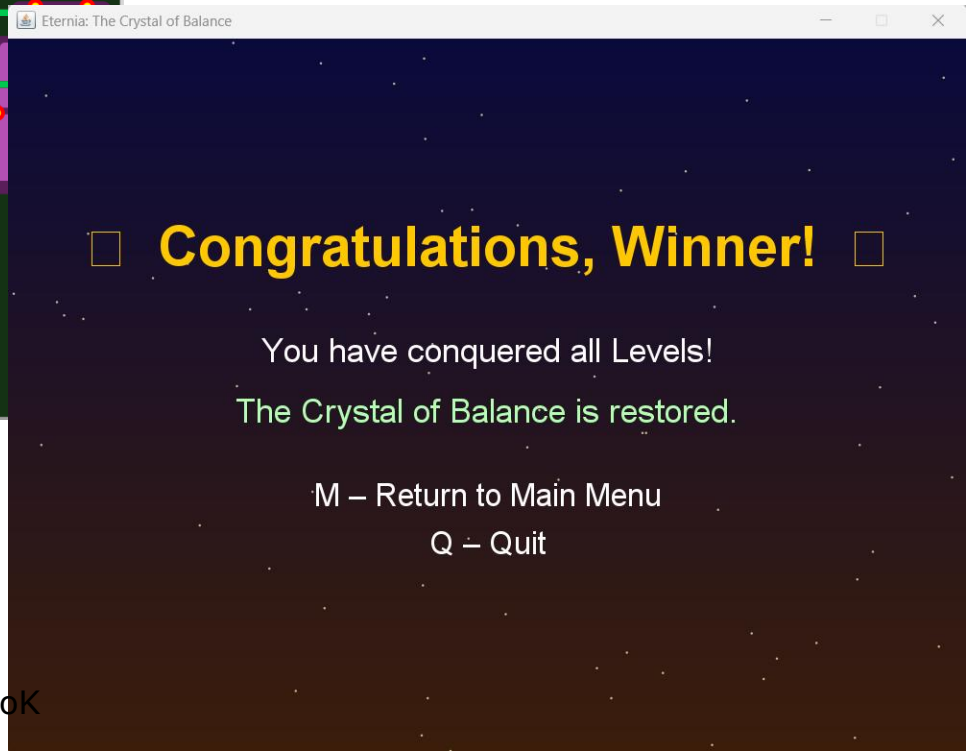


4. 9. Games Design



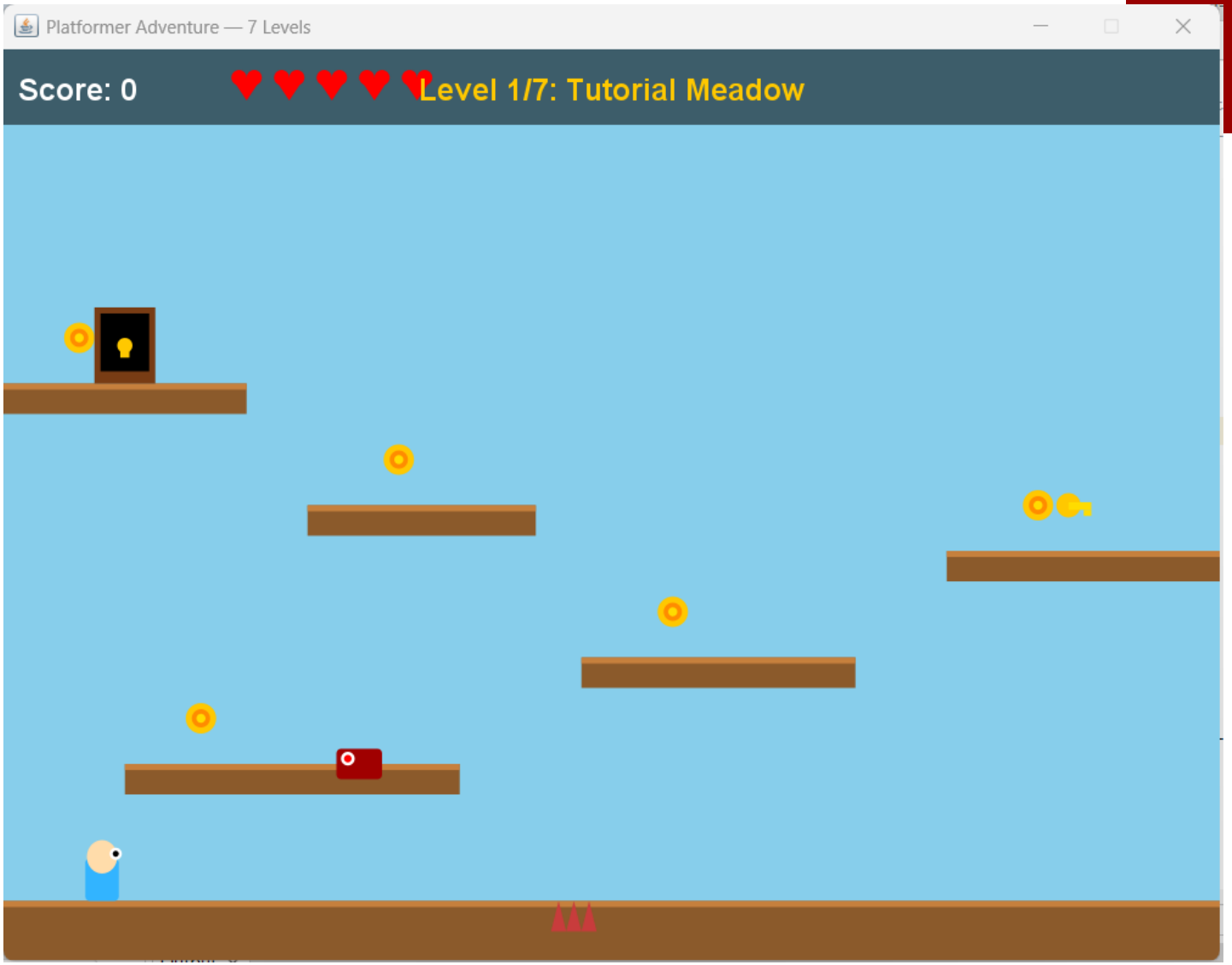
4. 9. Games Design



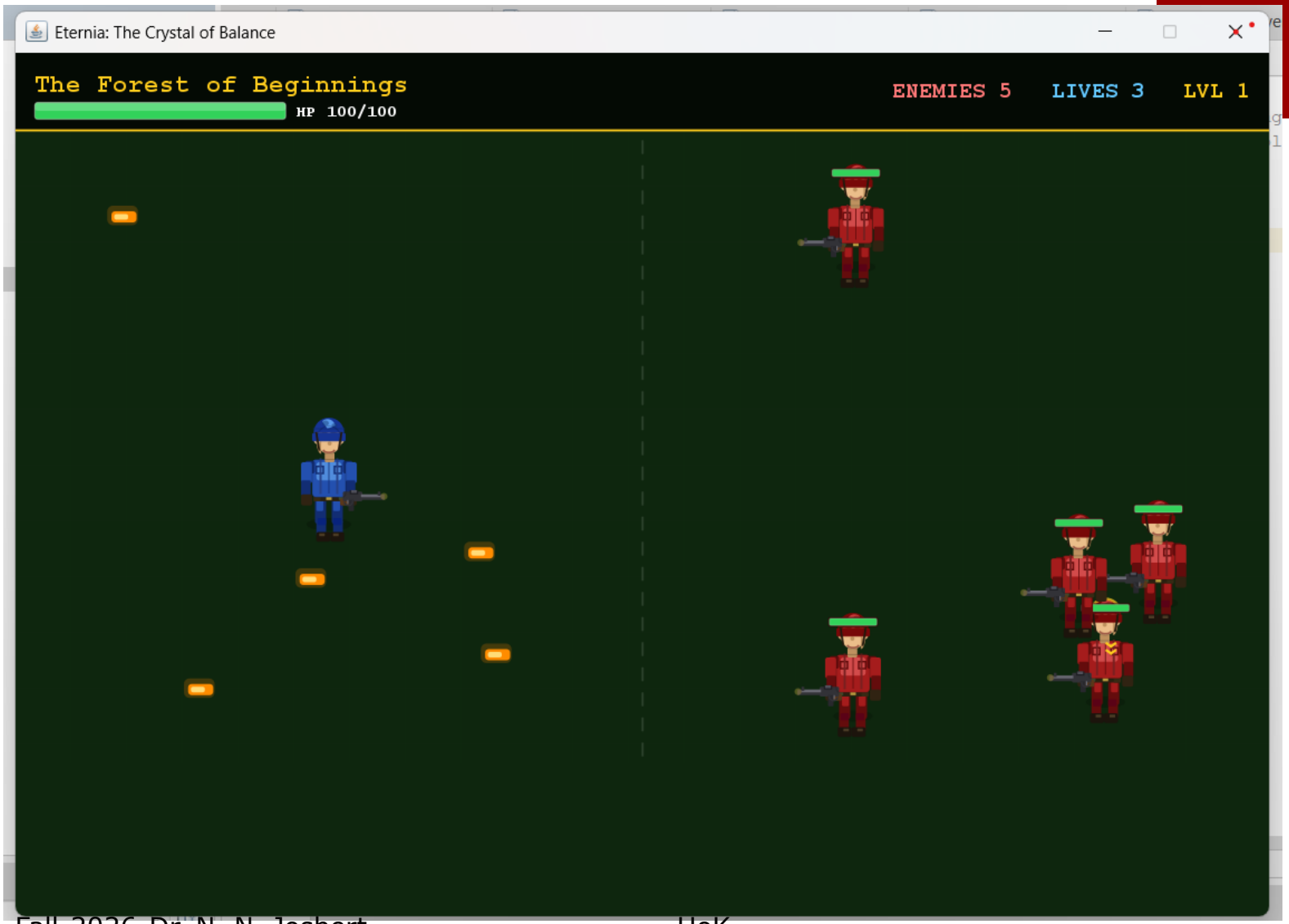


4. 9. Games Design

4. 9. Games Design



4. 9. Games Design



4. 9. Games Design